第七章

图

主要内容

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 应用举例

7.5 有向无环图及其应用

§7.1 图的定义和术语

- 图的结构定义:

图是由一个顶点集V 和一个弧集R构成的数据结构。

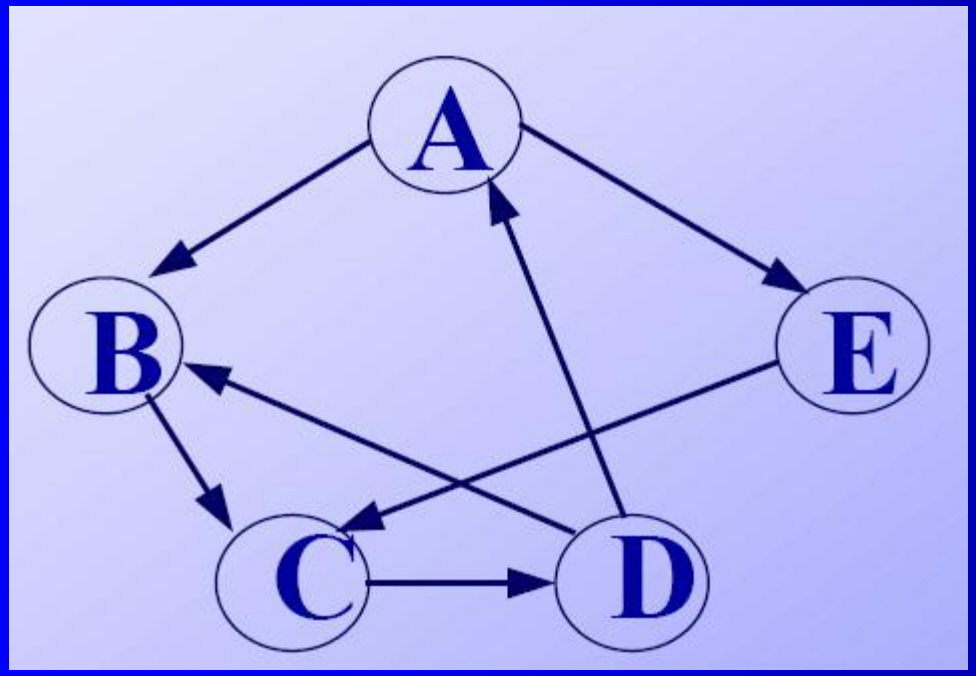
Graph = (V , R)

其中，VR＝{<v,w>| v,w∈V 且P(v,w)}

<v,w>表示从v 到w 的一条弧，并称v 为弧头，w 为弧尾。

谓词P(v,w) 定义了弧<v,w>的意义或信息。

- 由于“弧”是有方向的，因此称由顶点集和弧集构成的图为有向图。
- 例如：G1 = (V1, VR1)



其中

V1={A, B, C, D, E}

VR1={<A,B>, <A,E>, <B,C>, <C,D>, <D,B>, <D,A>, <E,C> }

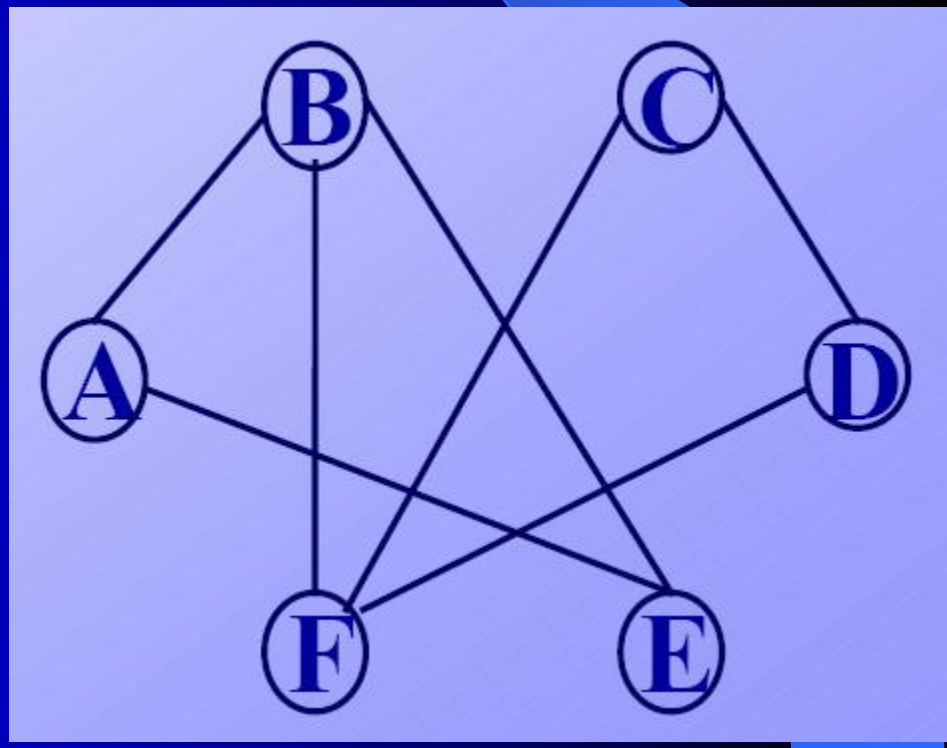
- 若<v, w>∈VR 必有<w, v>∈VR,则称(v,w) 为顶点v和顶点w 之间存在一条边。

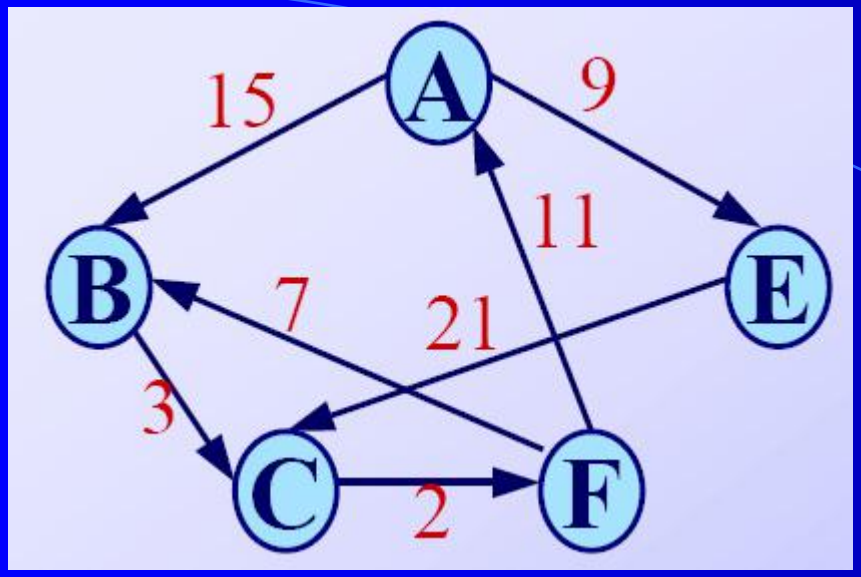
由顶点集和边集构成的图称作无向图。

例如: G2=(V2,VR2)

V2={A, B, C, D, E, F}

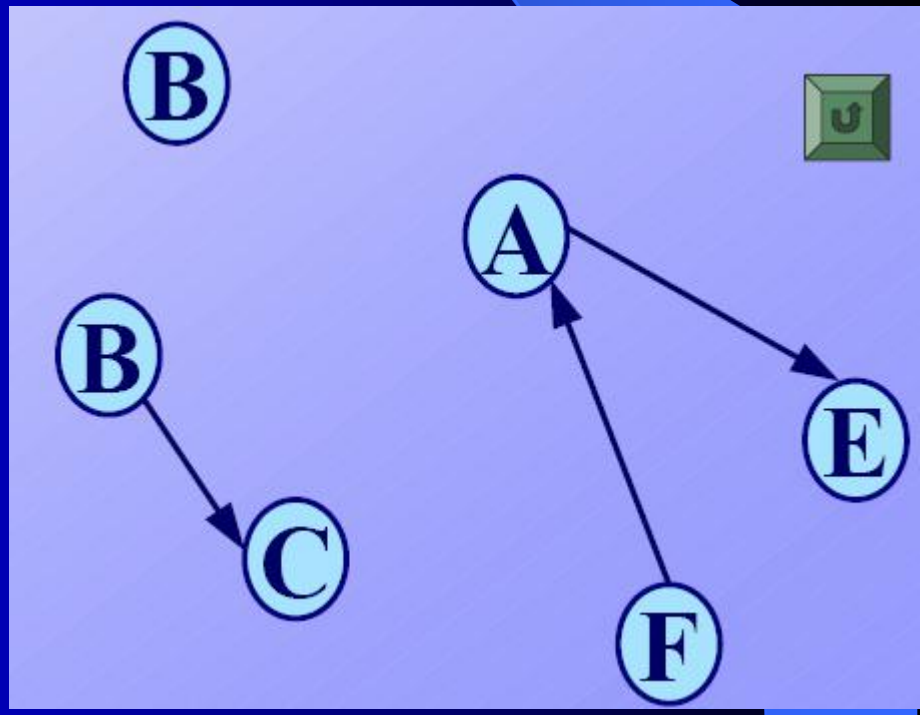
VR2={(A,B), (A,E), (B,E), (C,D), (D,F), (B,F), (C,F) }





弧或边带权的图分别称作有向网或无向网。

设图G=(V,{VR}) 和图G′ =(V′,{VR′ }), 且V′ ⊆V, VR′ ⊆VR, 则称G′ 为G 的子图



- 假设图中有n 个顶点，e 条边，则对于无向图e的取值范围是0到e=n(n-1)/2
- 含有e=n(n-1)/2 条边的无向图称作完全图；
- 含有e=n(n-1) 条弧的有向图称作有向完全图；
- 若边或弧的个数e<nlogn，则称作稀疏图，否则称作稠密图。

假若顶点v 和顶点w 之间存在一条边，则称顶点v 和w 互为邻接点，

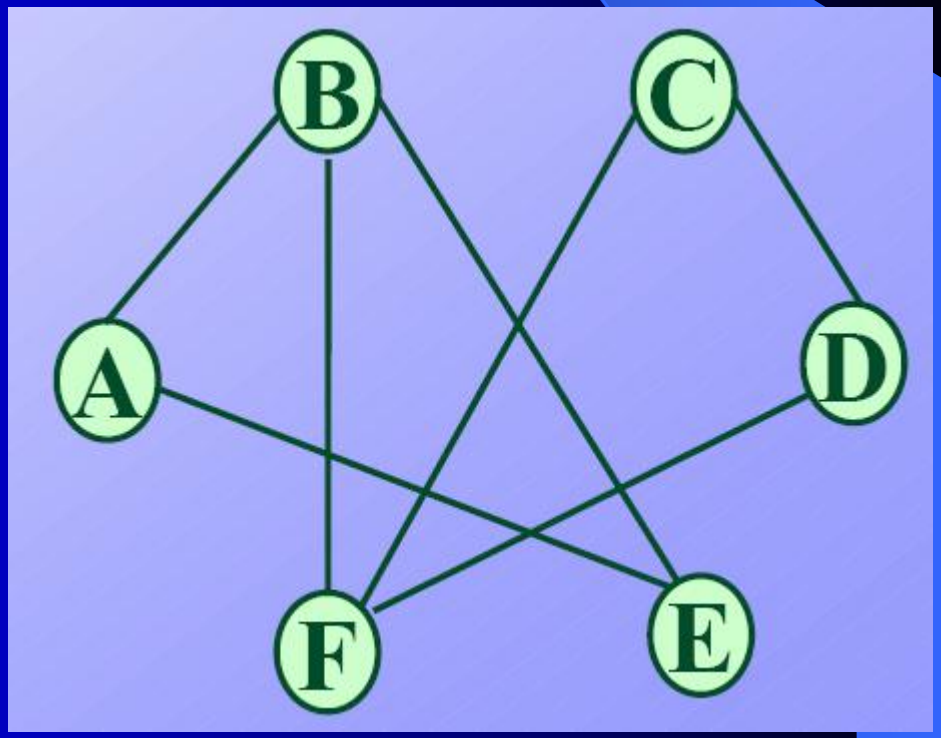
边(v,w) 和顶点v 和w 相关联。

和顶点v 关联的边的数目定义为顶点的度。

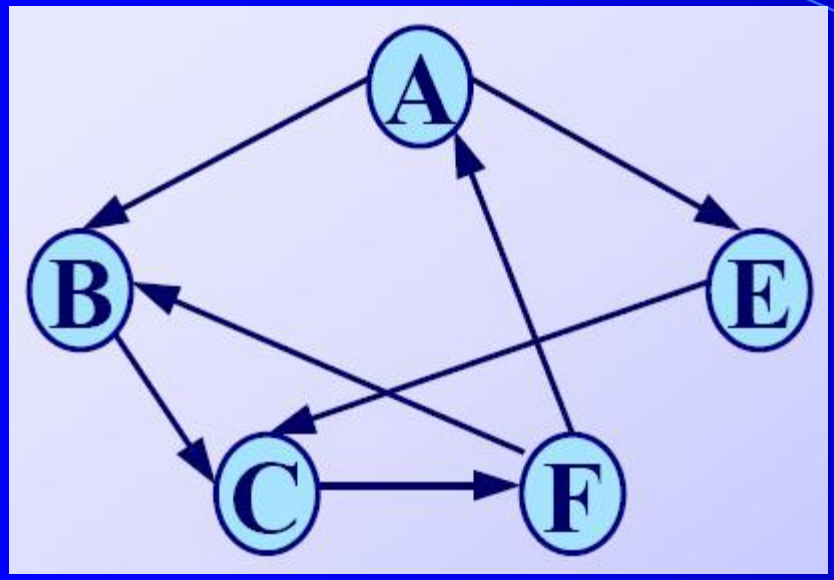
例如:

TD(B) = 3

TD(A) = 2



对有向图来说



顶点的出度: 以顶点v为弧尾的弧的数目；

顶点的入度: 以顶点v为弧头的弧的数目。

例如:

ID(B) = 2

OD(B) = 1

TD(B) = 3

顶点的度(TD)=出度(OD)+入度(ID)

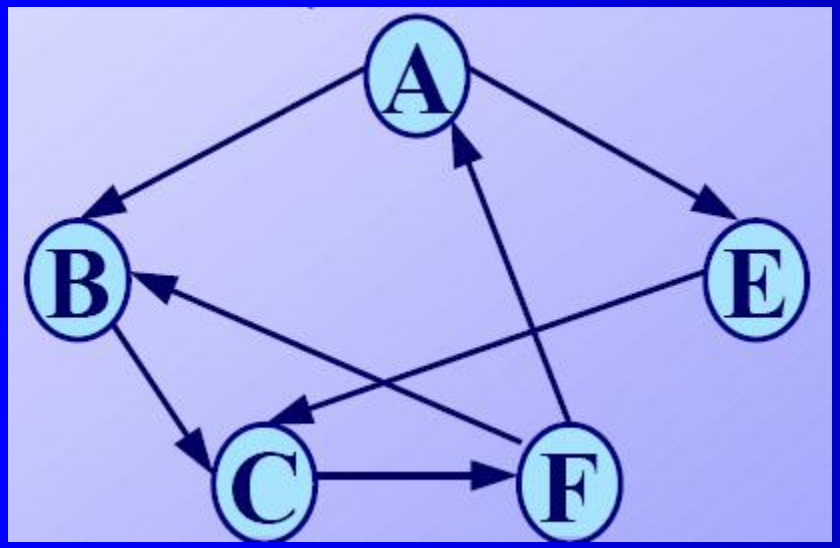
设图G=(V,{VR})中的一个顶点序列

{ $u=v_{i,0},v_{i,1}, \cdots, v_{i,m}=w$}中，$(v_{i,j-1},v_{i,j})\in VR$

$1\leqslant j\leqslant m$,则称从顶点u 到顶点w 之间存在一条路径。路径上边的数目称作路径长度。

如:长度为3的路径

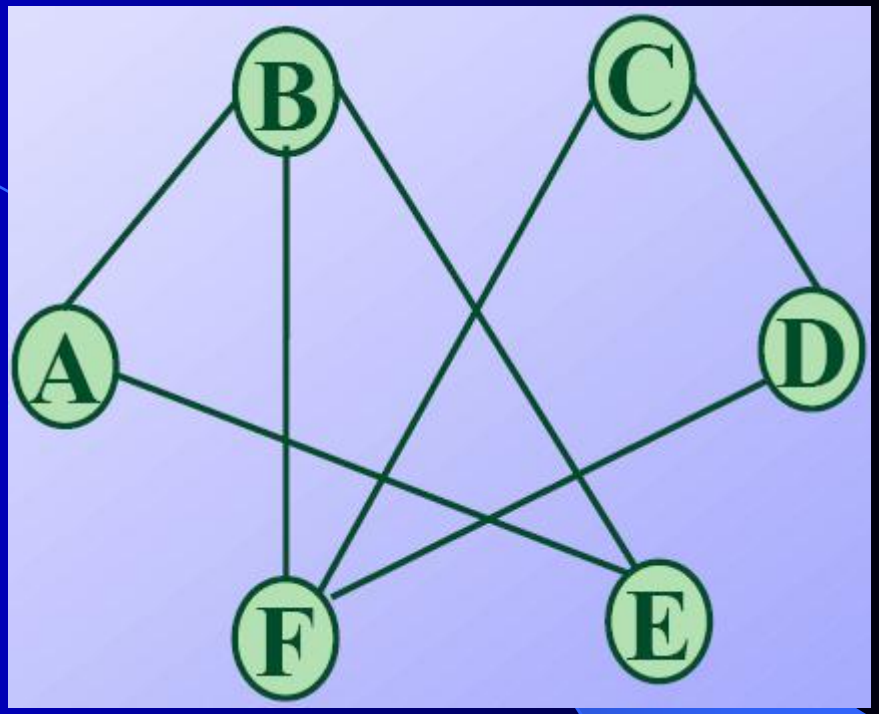
{A,B,C,F}

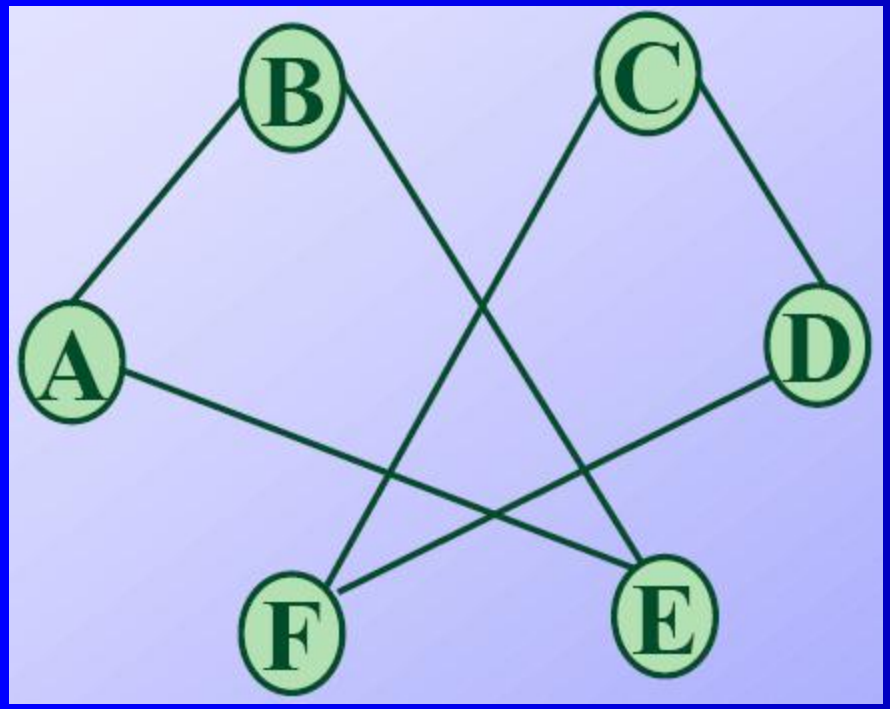


简单路径:序列中顶点不重复出现的路径。

简单回路:序列中第一个顶点和最后一个顶点相同的路径。

若图G中任意两个顶点之间都有路径相通，则称此图为连通图

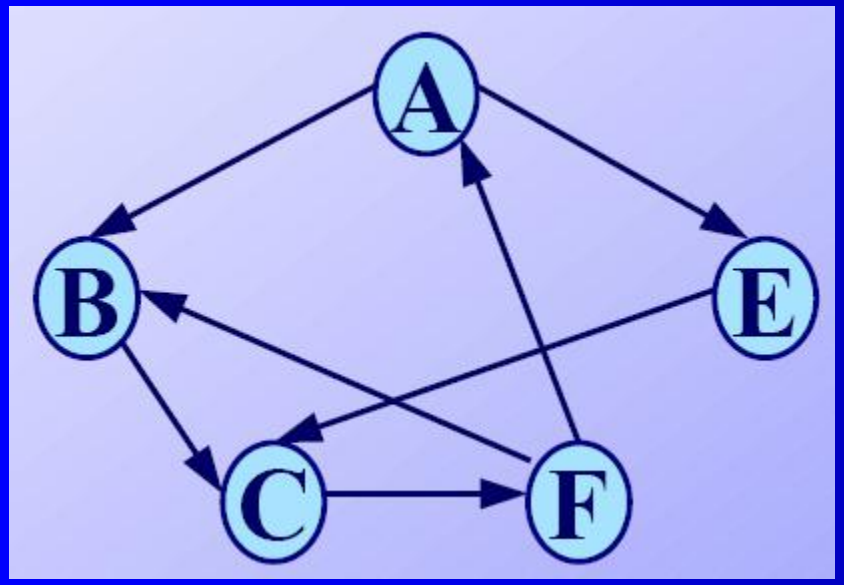


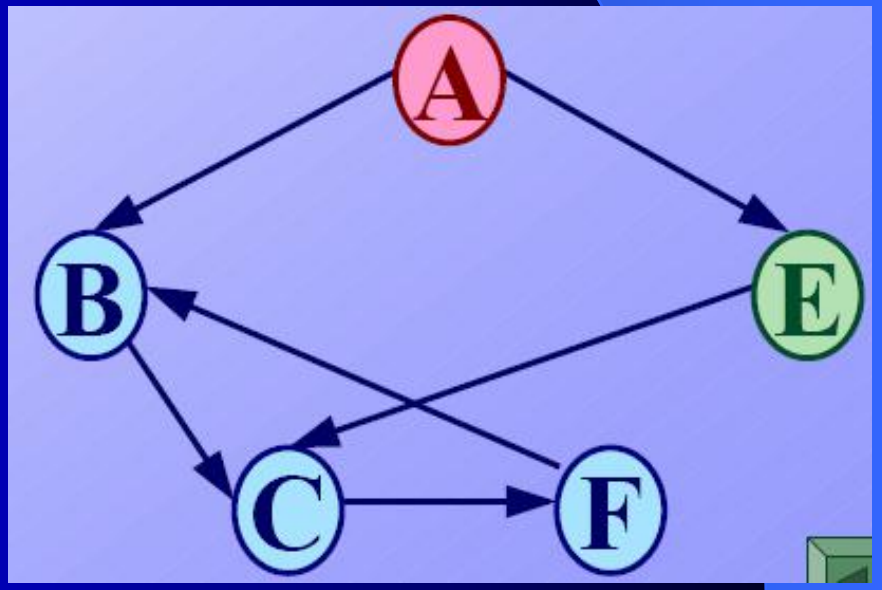


若无向图为非连通图，则图中各个极大连通子图称作此图的连通分量。

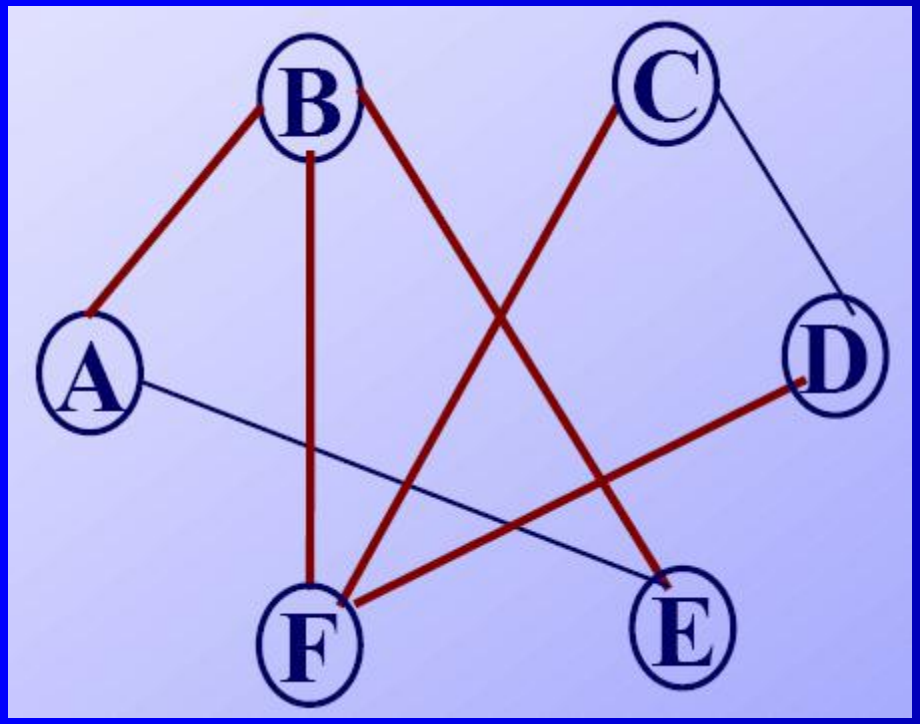
对有向图，若任意两个顶点之间都存在一条有向路径，则称此有向图为强连通图。

否则，其各个强连通子图称作它的强连通分量。





假设一个连通图有n 个顶点和e 条边，其中n-1 条边和n 个顶点构成一个极小连通子图，称该极小连通子图为此连通图的生成树。



对非连通图，则称由各个连通分量的生成树的集合为此非连通图的生成森林。

抽象数据类型图的定义

ADT Graph {

数据对象 V:

V是具有相同特性的数据元素的集合，称为顶点集。

数据关系 R:

R = { VR }

VR ={ < v, w > | v,w∈ V且P(v, w), < v, w >表示从v 到w 的弧 ，谓词P (v, w)定义了弧< v, w > 的意义或信息 }

基本操作

结构的建立和销毁

对顶点的访问操作

插入或删除顶点

插入和删除弧

对邻接点的操作

遍历

结构的建立和销毁

CreatGraph(&G, V, VR):

// 按定义(V, VR) 构造图

DestroyGraph(&G):

// 销毁图

对顶点的访问操作

LocateVex(G, u);

// 若G中存在顶点u，则返回该顶点在

// 图中“位置”；否则返回其它信息。

GetVex(G, v); // 返回 v 的值。

PutVex(&G, v, value);

// 对 v 赋值value。

对邻接点的操作

FirstAdjVex(G, v);

// 返回 v 的“第一个邻接点”。若该顶点
//在 G 中没有邻接点，则返回“空”。

NextAdjVex(G, v, w);

// 返回 v 的（相对于 w 的）“下一个邻接
// 点”。若 w 是 v 的最后一个邻接点，则
// 返回“空”。

插入或删除顶点

InsertVex(&G, v);

//在图G中增添新顶点v。

DeleteVex(&G, v);

// 删除G中顶点v及其相关的弧。

插入和删除弧

InsertArc(&G, v, w);

// 在G中增添弧<v,w>，若G是无向的，

//则还增添对称弧<w,v>。

DeleteArc(&G, v, w);

//在G中删除弧<v,w>，若G是无向的，

//则还删除对称弧<w,v>。

遍　历

DFSTraverse(G, v, Visit());

//从顶点v起深度优先遍历图G，并对每

//个顶点调用函数Visit一次且仅一次。

BFSTraverse(G, v, Visit());

//从顶点v起广度优先遍历图G，并对每

//个顶点调用函数Visit一次且仅一次。

§7.2 图的存储结构

- 图的结构较复杂，任意两个顶点间都可能存在联系，因而图的存储方法也很多，应根据具体的应用和施加的操作选择图的存储表示法

1. 图的数组(邻接矩阵)存储表示
2. 图的邻接表存储表示
3. 有向图的十字链表存储表示
4. 无向图的邻接多重表存储表示

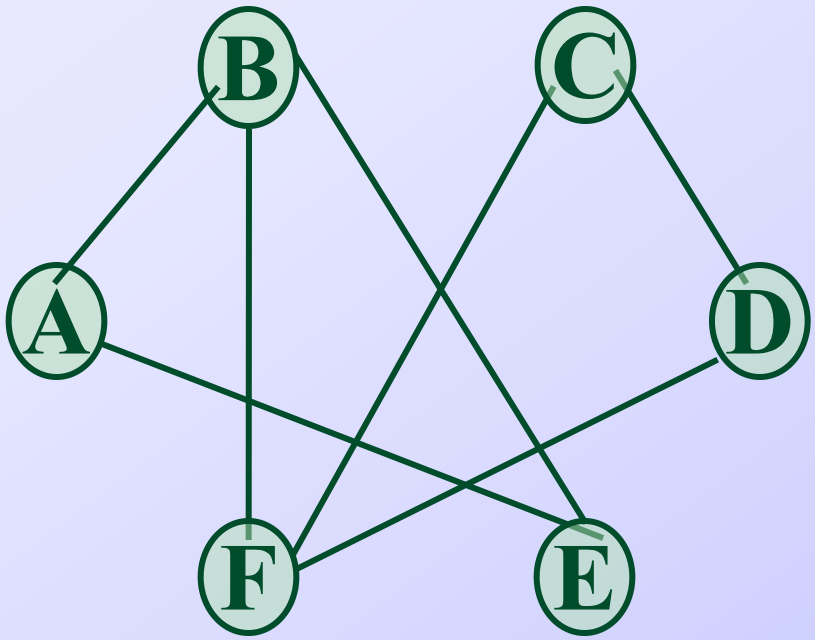
1. 图的数组(邻接矩阵)存储表示

数组表示法是用两个数组分别存储数据元素（顶点）的信息和数据元素之间的关系（边或弧）的信息。

邻接矩阵定义:矩阵的元素为

$$A_{ij}=\begin{cases} 0 & (i,j)\notin VR \\ 1 & (i,j)\in VR \end{cases}$$

无向图的邻接矩阵

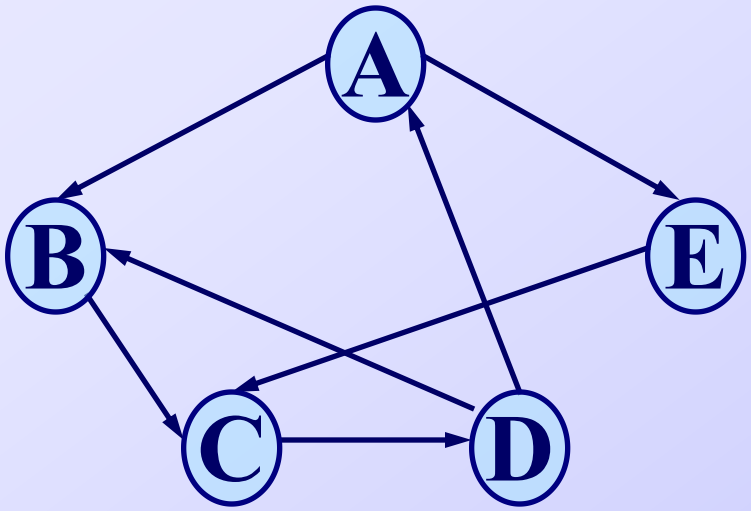


	A	B	C	D	E	F
A	0	1	0	0	1	0
B	1	0	0	0	1	1
C	0	0	0	1	0	1
D	0	0	1	0	0	1
E	1	1	0	0	0	0
F	0	1	1	1	0	0

无向图的邻接矩阵为对称矩阵。

顶点v_i的度是邻接矩阵第i行的元素之和。

有向图的邻接矩阵



	A	B	C	D	E
A	0	1	0	0	1
B	0	0	1	0	0
C	0	0	0	1	0
D	1	1	0	0	0
E	0	0	1	0	0

有向图的邻接矩阵为非对称矩阵。

第 i 行元素之和为顶点v_i的出度。

第 j 列元素之和为顶点v_j的入度。

特点:

- 无向图的邻接矩阵一定是一对称矩阵
- 无向图的邻接矩阵的第i行(或第i列)非零元素个数为第i个顶点的度$D(v_i)$。
- 有向图的邻接矩阵的第i行非零元素个数为第i个顶点的出度$OD(v_i)$，第i列非零元素个数就是第i个顶点的入度$ID(v_i)$。
- 邻接矩阵表示图，很容易确定图中任意两个顶点之间是否有边相连。

```
typedef struct ArcCell { // 弧的定义
    VRType  adj;  // VRType是顶点关系类型。
                  // 对无权图，用1或0表示相邻否；
                  // 对带权图，则为权值类型。
    InfoType  *info;  // 该弧相关信息的指针
} ArcCell,
    AdjMatrix[MAX_VERTEX_NUM]
                  [MAX_VERTEX_NUM];
```

```
typedef struct {        // 图的定义
    VertexType          // 顶点信息
                vexs[MAX_VERTEX_NUM];
    AdjMatrix    arcs;      // 弧的信息
    int    vexnum, arcnum;   // 顶点数，弧数
    GraphKind   kind;     // 图的种类标志
} MGraph;
```

```
Status CreateGraph(MGraph  &G) {
// 采用数组（邻接矩阵）表示法，构造图G
scanf(&G.kind);
switch(G.kind) {
  case  DG: return  CreateDG(G);
  case  DN: return  CreateDN(G);
  case  UDG: return  CreateUDG(G);
  case  UDN: return  CreateUDN(G);
  default: return  ERROR;   }
}
```

```
Status CreateUDN(MGraph  &G) {
//采用数组（邻接矩阵）表示法构造无向网G
scanf(&G.vexnum,&G.arcnum,&IncInfo);
//IncInfo为0则各弧不含其它信息
for(i=0;i<G.vexnum;++i) scanf(&G.vexs[i]);
//构造顶点向量
for(i=0;i<G.vexnum;++i)  // 初始化邻接矩阵
   for(j=0;j<G.vexnum;++j)
      G.arcs[i][j]={INFINITY,NULL};
```

```
for(k=0;k<G.arcnum;++k)  { // 构造邻接矩阵
   scanf(&v1,&v2,&w);
   //输入一条边依附的顶点及权值
   i=LocateVex(G,v1); j=LocateVex(G,v2);
   //确定v1和v2在G中位置
   G.arcs[i][j].adj=w;  //弧<v1,v2>的权值
   if(IncInfo) Input(*G.arcs[i][j].info);
   G.arcs[j][i]= G.arcs[i][j];
   // 置<v1,v2>的对称弧<v2,v1>   }
return OK;
}
```

2. 图的邻接表存储表示

由顺序存储的顶点表和n个链式存储的边表两部分组成：

1 顺序存储的顶点表存放顶点本身的数据信息，表中每个表目对应于图的一个顶点，包括两个字段：

- ✓ 顶点字段(data)存放顶点v_i的信息
- ✓ 指针字段(firstarc)存放与v_i相关联的边表中的第一个边结点的地址

data	firstarc

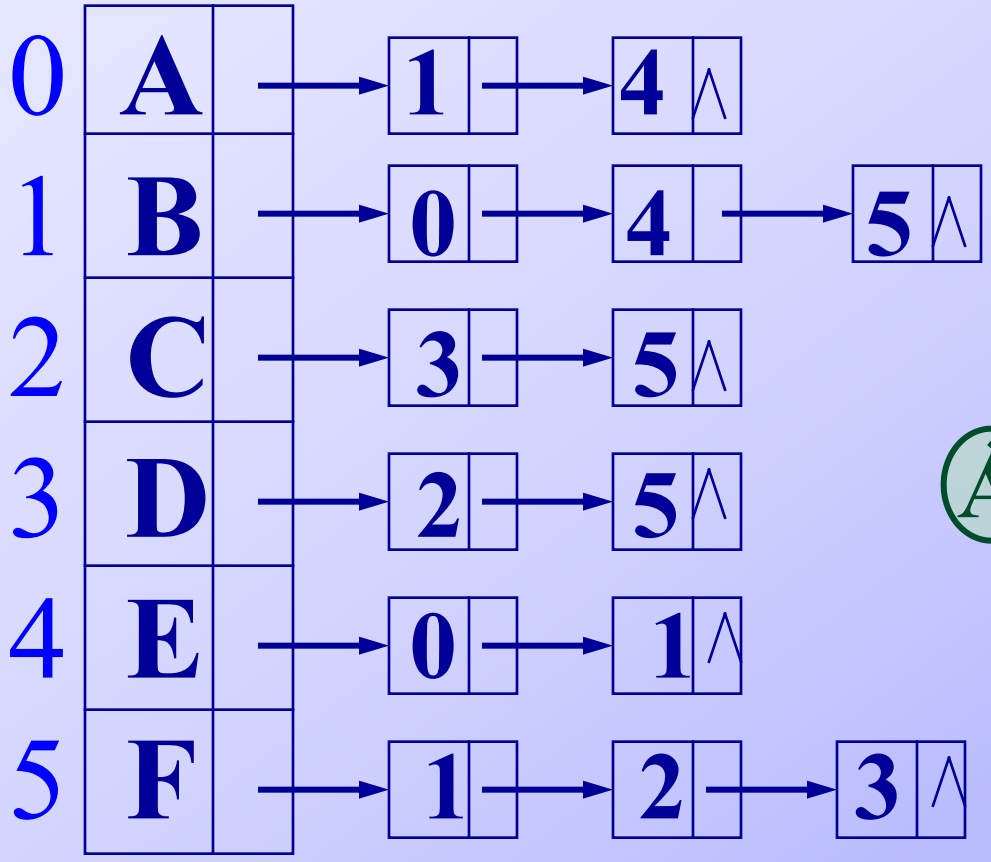
2　n个链式存储的边表，边表中每个边结点包括三个字段：

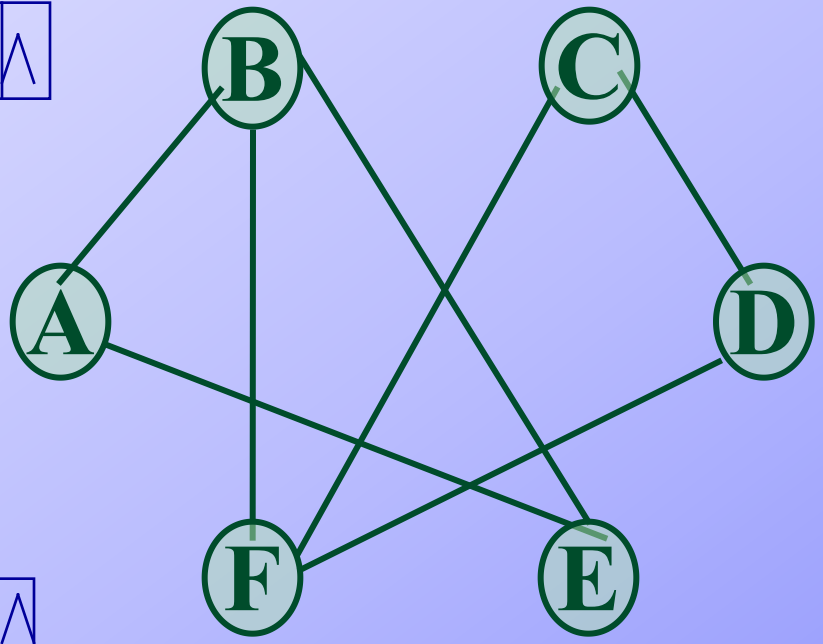
- 相邻顶点字段(adjvex)存放通过本边与顶点v_i相邻接的顶点v_j在顶点表中的位置j
- 信息字段(info)存放边结点所代表的边的信息
- 链字段(nextarc)指向边表的下一个边结点

adjvex	nextarc	info

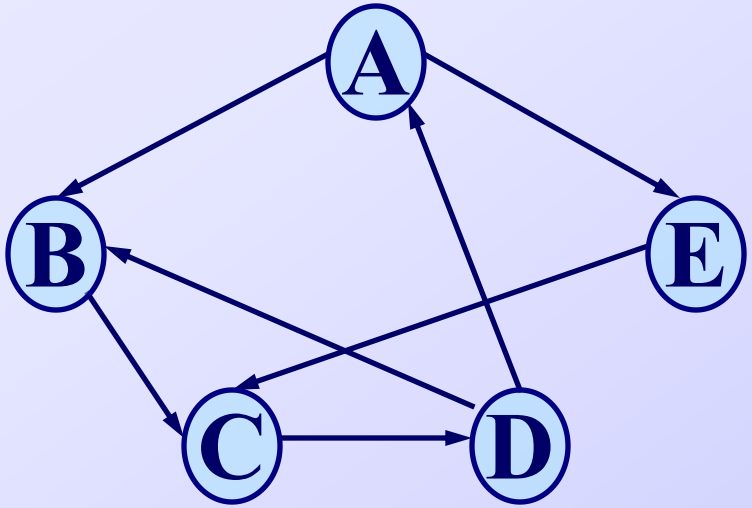
图的邻接表存储表示

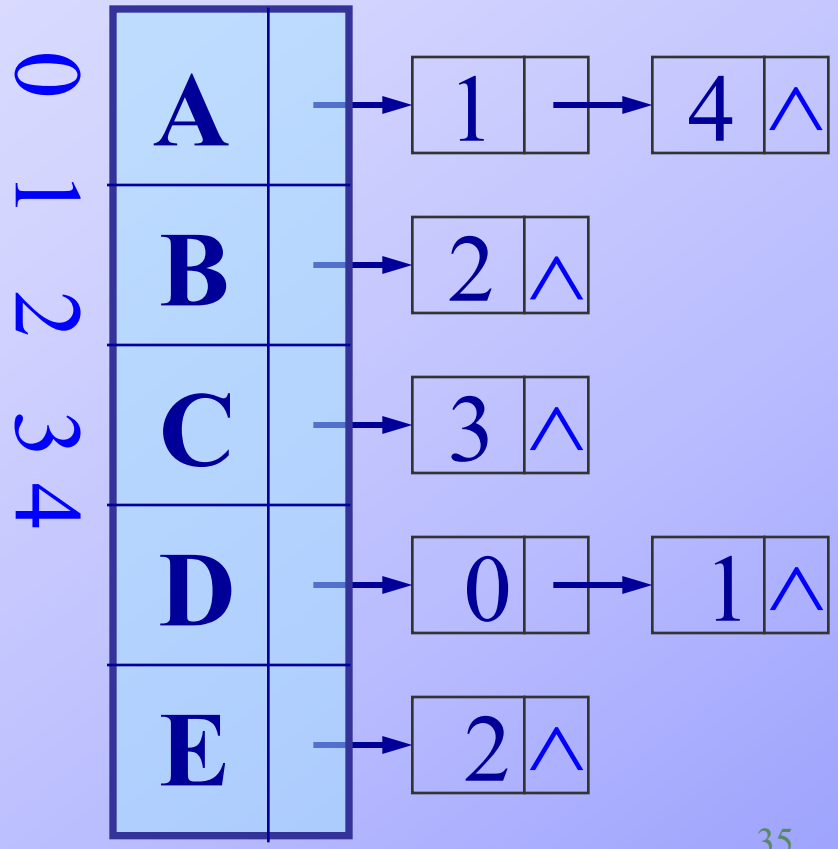
第i个单链表中的结点表示依附于顶点v_i的边（对有向图是以顶点v_i为尾的弧）。





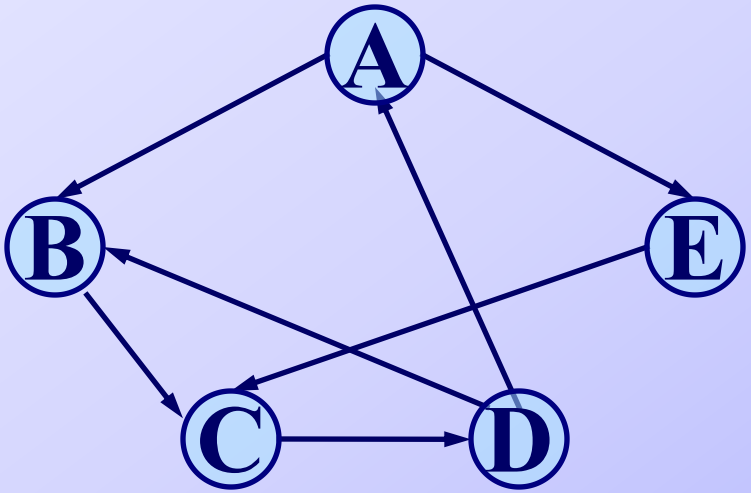
有向图的邻接表



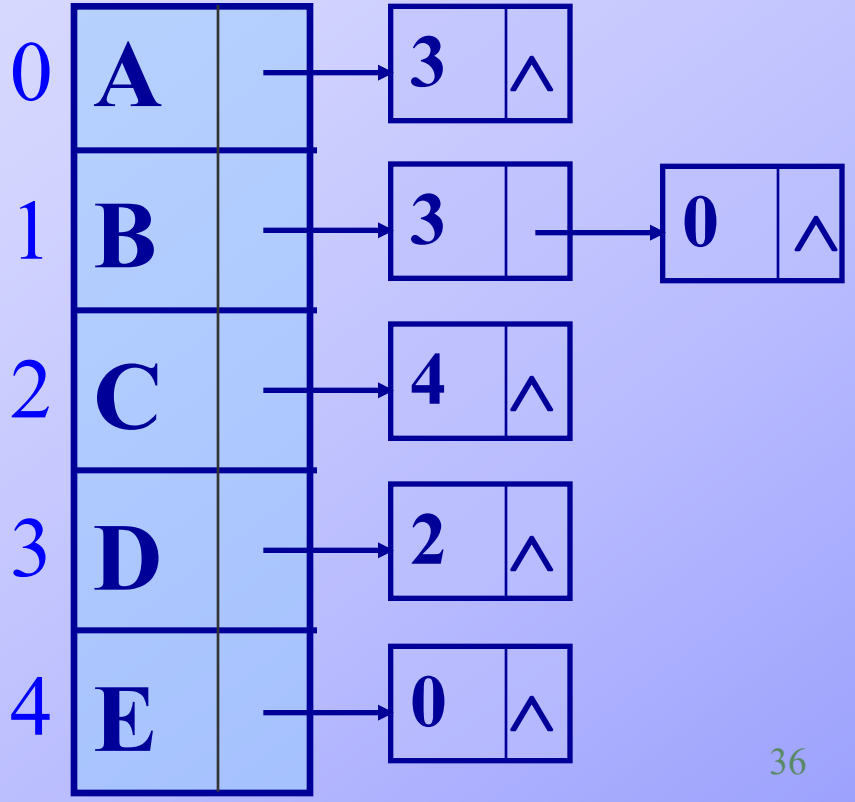


可见，在有向图的邻接表中不易找到指向该顶点的弧。

有向图的逆邻接表



在有向图的逆邻接表中，对每个顶点，链接的是指向该顶点的弧。



弧的结点结构

adjvex	nextarc	info

```
typedef struct ArcNode {
  int        adjvex;   // 该弧所指向的顶点的位置
  struct ArcNode  *nextarc;
                       // 指向下一条弧的指针
  InfoType   *info;   // 该弧相关信息的指针
} ArcNode;
```

顶点的结点结构

data	firstarc

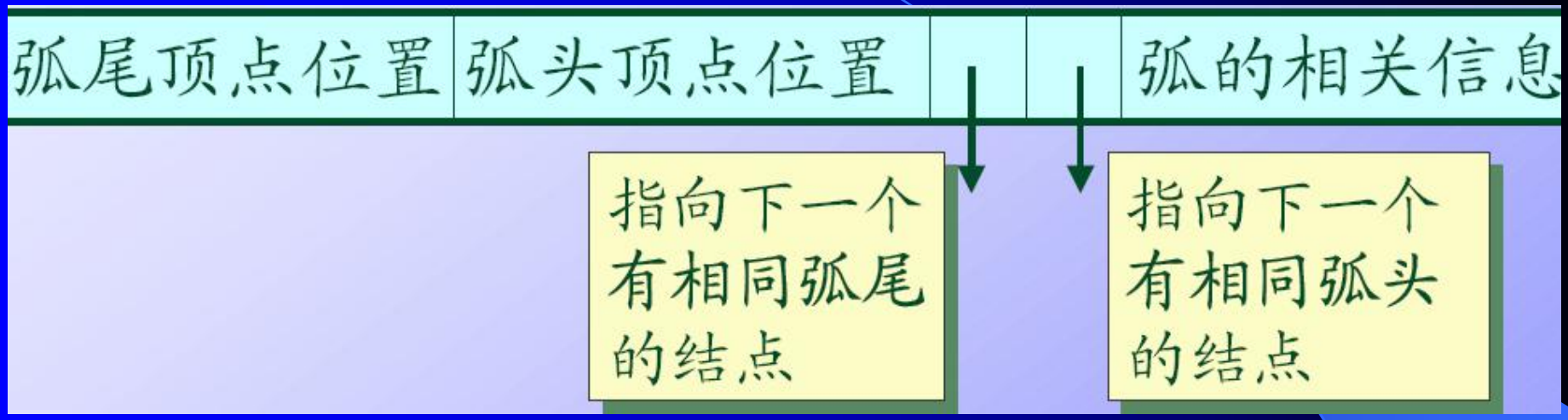
```
typedef struct VNode {
  VertexType  data;   // 顶点信息
  ArcNode  *firstarc;
                  // 指向第一条依附该顶点的弧
  } VNode, AdjList[MAX_VERTEX_NUM];
```

图的结构定义

```
typedef struct {
    AdjList  vertices;
    int      vexnum, arcnum;
    int      kind;          // 图的种类标志
} ALGraph;
```

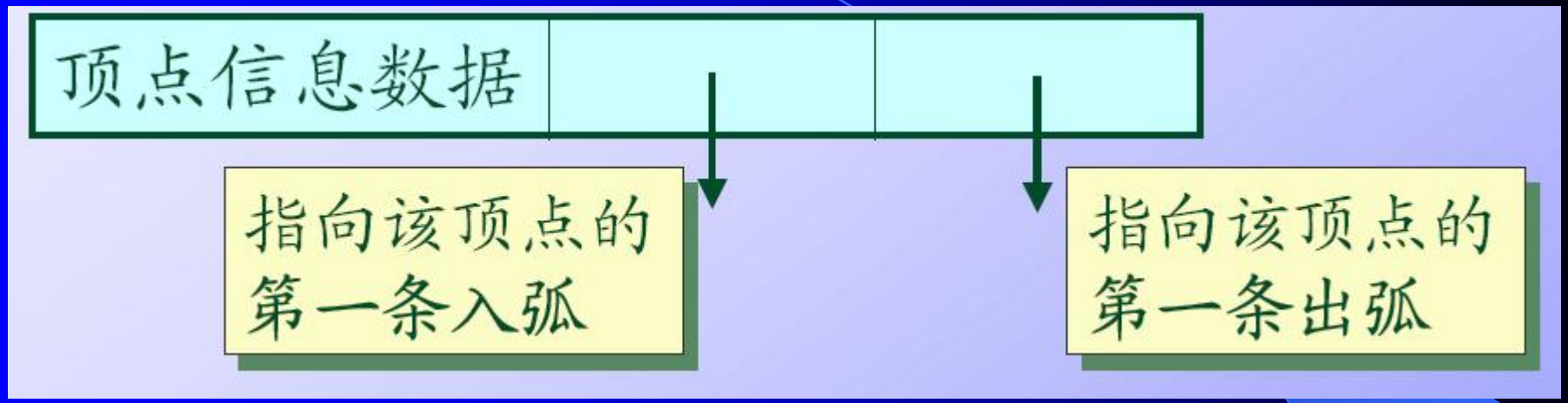
3. 有向图的十字链表存储表示

弧的结点结构




```
typedef struct ArcBox { // 弧的结构表示
    int tailvex, headvex; InfoType *info;
    struct ArcBox *hlink, *tlink;
} VexNode;
```

顶点的结点结构




```
typedef struct VexNode { // 顶点的结构表示
    VertexType data;
    ArcBox *firstin, *firstout;
} VexNode;
```

有向图的结构表示(十字链表)

```
typedef struct {
    VexNode xlist[MAX_VERTEX_NUM];
            // 顶点结点(表头向量)
    int vexnum, arcnum;
            //有向图的当前顶点数和弧数
} OLGraph;
```

```
Status CreateDG(OLGraph  &G)  {
// 采用十字链表存储表示，构造有向图G
scanf(&G.vexnum,&G.arcnum,&IncInfo);
for(i=0;i<G.vexnum;++i)  { //构造表头向量
   scanf(G.xlist[i].data);
   G.xlist[i].firstin=NULL;
   G.xlist[i].firstout=NULL;   }
for(k=0;k<G.arcnum;++k)  {
   scanf(&v1,&v2);
   i=Locate(G,v1); j=Locate(G,v2);
   p=(ArcBox *)malloc(sizeof(ArcBox));
   *p={i,j,G.xlist[j].firstin,G.xlist[i].firstout,NULL};
   G.xlist[j].firstin=G.xlist[i].firstout=p;
   // 完成在入弧和出弧链头的插入    }   }
```

4. 无向图的邻接多重表存储表示

在多重表中每一条边用一个结点表示，每一个顶点也用一个结点表示。

在邻接多重表中，所有依附于同一顶点的边串联在同一链表中，由于每条边依附于两个顶点，则每个边结点同时链接在两个链表中。

边的结构表示

```
typedef struct Ebox {
    VisitIf mark;      // 访问标记
    int ivex, jvex;
        //该边依附的两个顶点的位置
    struct EBox *ilink, *jlink;
        //分别指向依附这两个顶点的下一条边
    InfoType *info; // 该边信息指针
} EBox;
```

mark	ivex	ilink	jvex	jlink	info

顶点的结构表示

```
typedef struct VexBox {
    VertexType data;
    EBox *firstedge;
      // 指向第一条依附该顶点的边
} VexBox;
```

data	firstedge

无向图的结构表示

```
typedef struct {   // 邻接多重表
    VexBox adjmulist[MAX_VERTEX_NUM];
    int vexnum, edgenum;
} AMLGraph;
```

§7.3 图的遍历

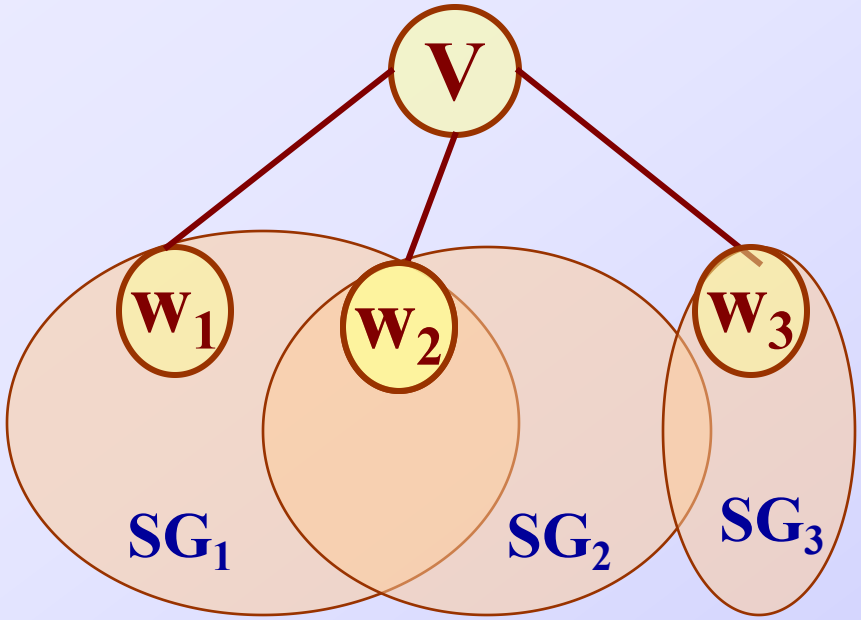
从图中某个顶点出发遍历图，访遍图中其余顶点，并且使图中的每个顶点仅被访问一次的过程。

- 深度优先搜索
- 广度优先搜索
- 遍历应用举例

一、深度优先搜索遍历图

连通图的深度优先搜索遍历

从图中某个顶点V_0 出发，访问此顶点，然后依次从V_0的各个未被访问的邻接点出发深度优先搜索遍历图，直至图中所有和V_0有路径相通的顶点都被访问到。



W_1、W_2和W_3 均为 V 的邻接点，SG_1、SG_2 和 SG_3 分别为含顶点 W_1、W_2和W_3 的子图。

访问顶点 V：

for (W_1、W_2、W_3)

若该邻接点W未被访问，

则从它出发进行深度优先搜索遍历。

从上页的图解可见深度优先搜索遍历连通图的过程类似于树的先根遍历。

如何判别V的邻接点是否被访问?

解决的办法是：为每个顶点设立一个“访问标志 visited[w]”。

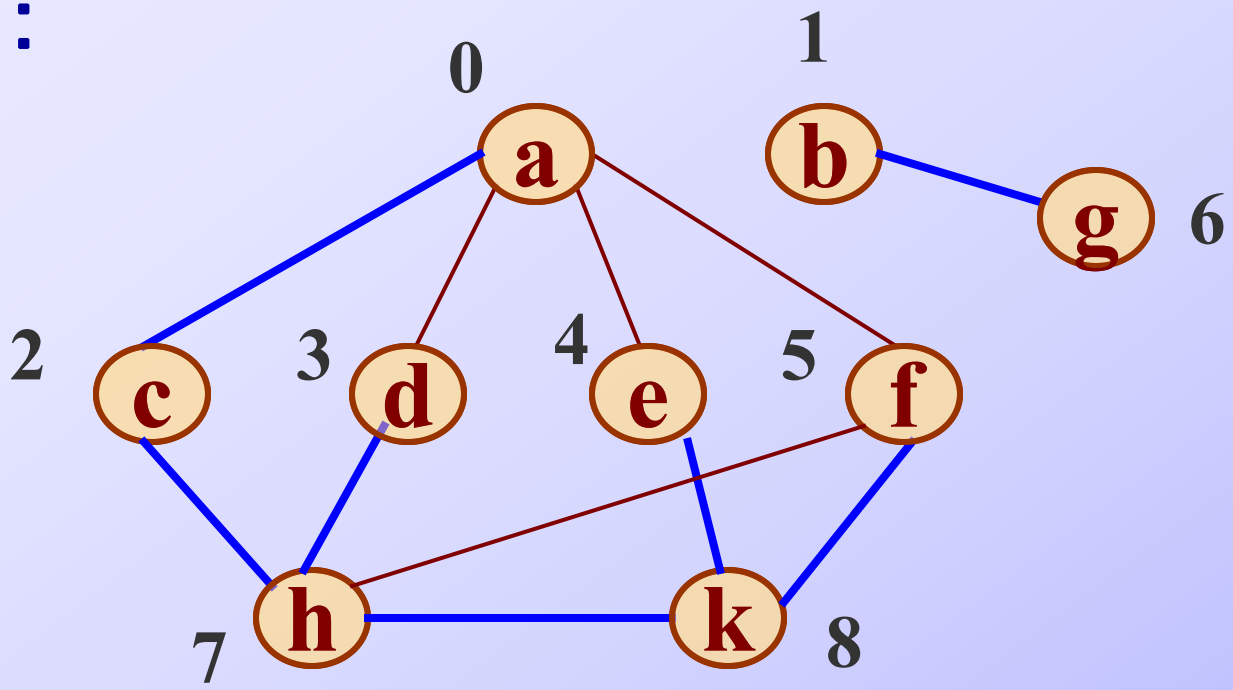
```
void DFS(Graph G, int v) {
  // 从顶点v出发，深度优先搜索遍历连通图 G
   visited[v] = TRUE;   VisitFunc(v);
   for(w=FirstAdjVex(G, v);
         w!=0; w=NextAdjVex(G,v,w))
      if (!visited[w])  DFS(G, w);
         // 对v的尚未访问的邻接顶点w
         // 递归调用DFS
} // DFS
```

非连通图的深度优先搜索遍历

首先将图中每个顶点的访问标志设为 FALSE,之后搜索图中每个顶点，如果未被访问，则以该顶点为起始点，进行深度优先搜索遍历，否则继续检查下一顶点。

```
void DFSTraverse(Graph G,
                        Status (*Visit)(int v)) {
  // 对图 G 作深度优先遍历。
  VisitFunc = Visit;
  for (v=0; v<G.vexnum; ++v)
    visited[v] = FALSE; // 访问标志数组初始化
  for (v=0; v<G.vexnum; ++v)
    if (!visited[v])  DFS(G, v);
          // 对尚未访问的顶点调用DFS
}
```

例如：

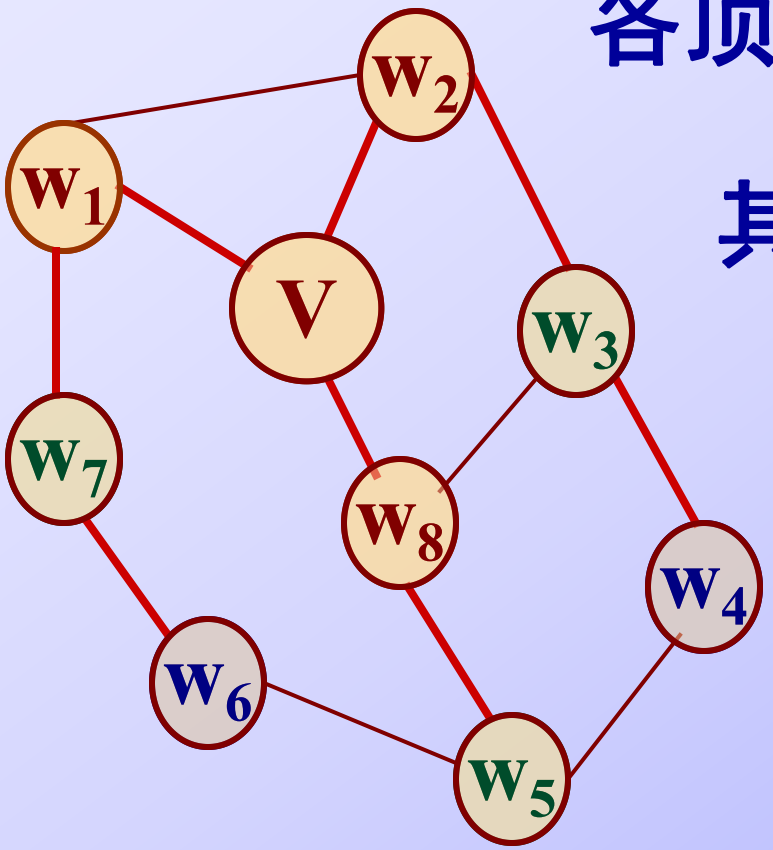


	0	1	2	3	4	5	6	7	8
访问标志：	T	T	T	T	T	T	T	T	T

访问次序：a c h d k f e b g

二、广度优先搜索遍历图

对连通图，从起始点V到其余各顶点必定存在路径。



其中，$V\text{->}w_1$, $V\text{->}w_2$, $V\text{->}w_8$ 的路径长度为1；

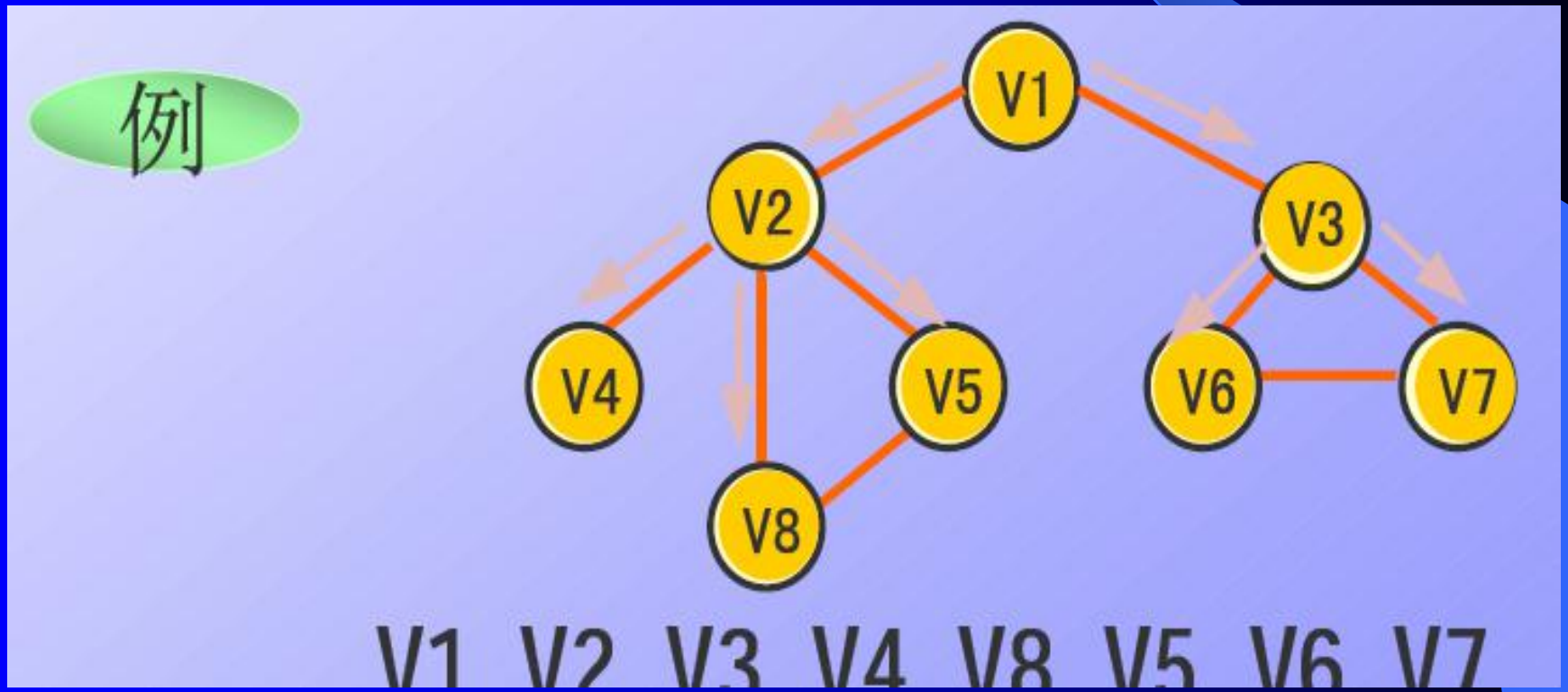
$V\text{->}w_7$, $V\text{->}w_3$, $V\text{->}w_5$ 的路径长度为2；

$V\text{->}w_6$, $V\text{->}w_4$ 的路径长度为3。

从图中的某个顶点V_0出发，并在访问此顶点之后依次访问V_0的所有未被访问过的邻接点，之后按这些顶点被访问的先后次序依次访问它们的邻接点，直至图中所有和V_0有路径相通的顶点都被访问到。

若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点，重复上述过程，直至图中所有顶点都被访问到为止。

广度优先搜索遍历图的过程是以v为起点，由近至远，依次访问和v有路径相联通且路径长度为1，2，3，…的顶点



```
void BFSTraverse(Graph G,
                    Status (*Visit)(int v)){
  for (v=0; v<G.vexnum; ++v)
     visited[v] = FALSE;  //初始化访问标志
  InitQueue(Q);        // 置空的辅助队列Q
  for ( v=0;  v<G.vexnum;  ++v )
    if ( !visited[v]) {          // v 尚未访问
      … …
  }
} // BFSTraverse
```

```
visited[v] = TRUE;  Visit(v);    // 访问v
EnQueue(Q, v);                   // v入队列
while (!QueueEmpty(Q))  {
  DeQueue(Q, u);
                    // 队头元素出队并置为u
  for(w=FirstAdjVex(G, u); w!=0;
                    w=NextAdjVex(G,u,w))
    if ( ! visited[w])  {
      visited[w]=TRUE;  Visit(w);
      EnQueue(Q, w); // 访问的顶点w入队列
    } // if
} // while
```

§7.4 应用举例

最小生成树

对于连通的无向图或强连通的有向图，从任一顶点出发遍历，可以访问到所有的顶点。遍历时经过的边加上所有顶点构成了图的一个连通子图，称为图的一棵生成树。

构造生成树的过程可以按深度优先遍历，也可以按广度优先遍历，遍历中记录访问的所有顶点以及经过的边，得到的是深度优先生成树或广度优先生成树，简称为DFS生成树或BFS生成树。

对于非连通的无向图和非强连通的有向图, 从任一顶点出发无法访问到所有的顶点，只能得到各连通分量的生成树所组成的生成树林。

图的生成树不唯一，从不同顶点出发，或从同一顶点出发，但遍历的路径不一样，则得到的生成树都不同。

最小生成树定义

对于网络，其生成树中的边也带权，将生成树各边的权值总和称为**生成树的权**，并把权值最小的生成树称为**最小生成树(Minimum Spanning Tree)**

问题：

假设要在 n 个城市之间建立通讯联络网，则连通 n 个城市只需要修建 $n-1$ 条线路，如何在最节省经费的前提下建立这个通讯网？

该问题等价于：

构造网的一棵最小生成树，即：

在 e 条带权的边中选取 n-1 条边（不构成回路），使“权值之和”为最小。

算法一：（普里姆算法）

算法二：（克鲁斯卡尔算法）

普里姆算法的基本思想:

取图中任意一个顶点 v 作为生成树的根，之后往生成树上添加新的顶点 w。在添加的顶点 w 和已经在生成树上的顶点v之间必定存在一条边，并且该边的权值在所有连通顶点 v 和 w 之间的边中取值最小。之后继续往生成树上添加顶点，直至生成树上含有 n-1 条边为止。

普鲁姆算法基本步骤：

设G=(V,E)为一个具有n个顶点的连通网络，

T=（U，TE）为构造的生成树。

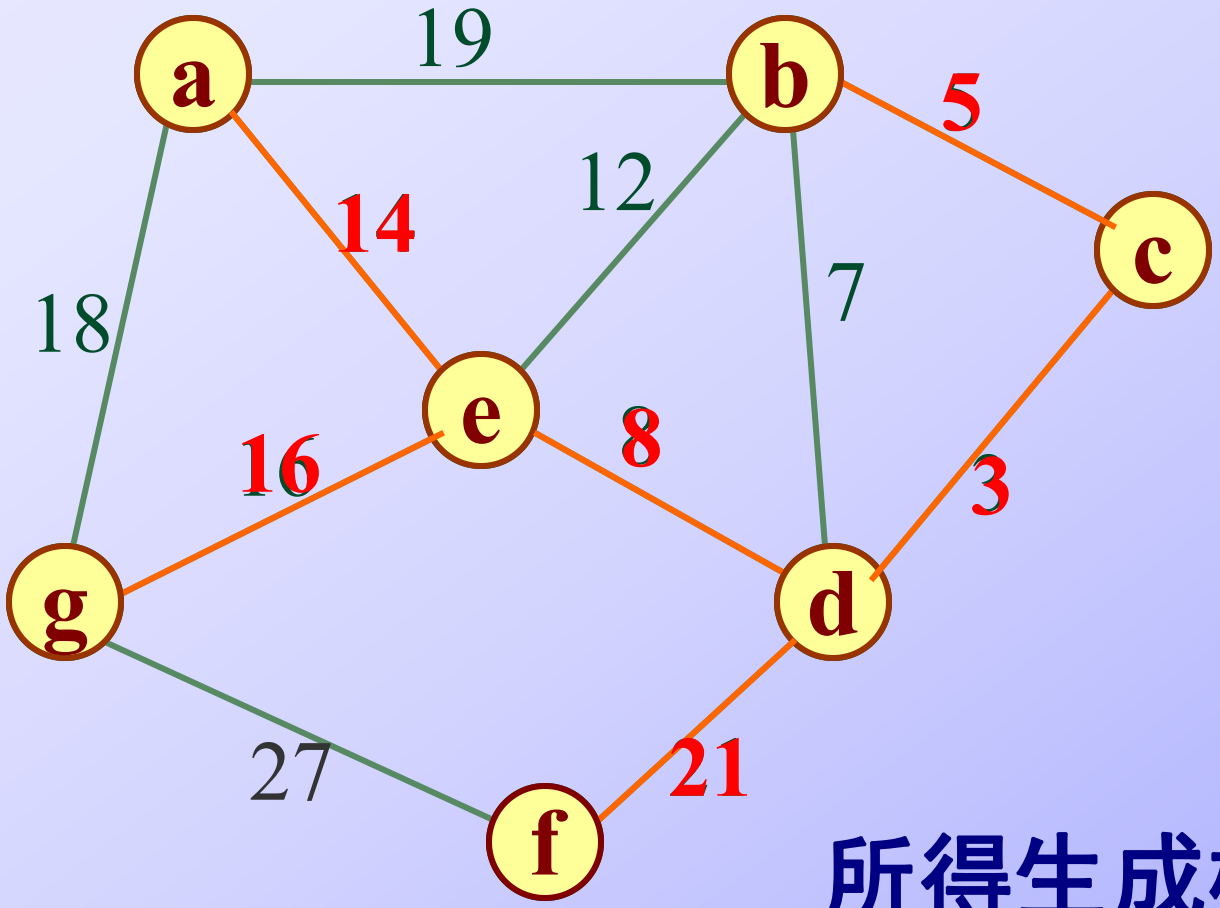
1）初始时，U={u_0}，TE=φ；

2）在所有u∈U 、v∈V-U的边（u,v）中选择一条权值最小的边，不妨设为（u,v）；

3）将v加入U；同时(u,v)加入TE，

4）重复2)、3)，直到U=V为止；

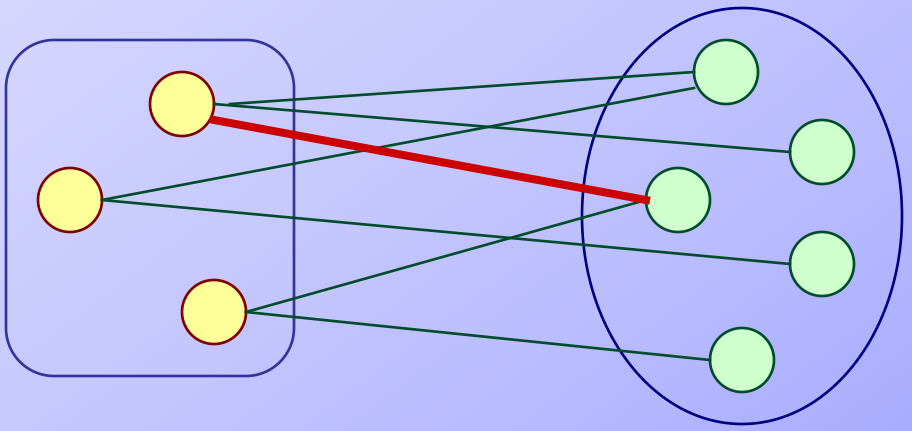
例如:



所得生成树权值和

= 14+8+3+5+16+21 = 67

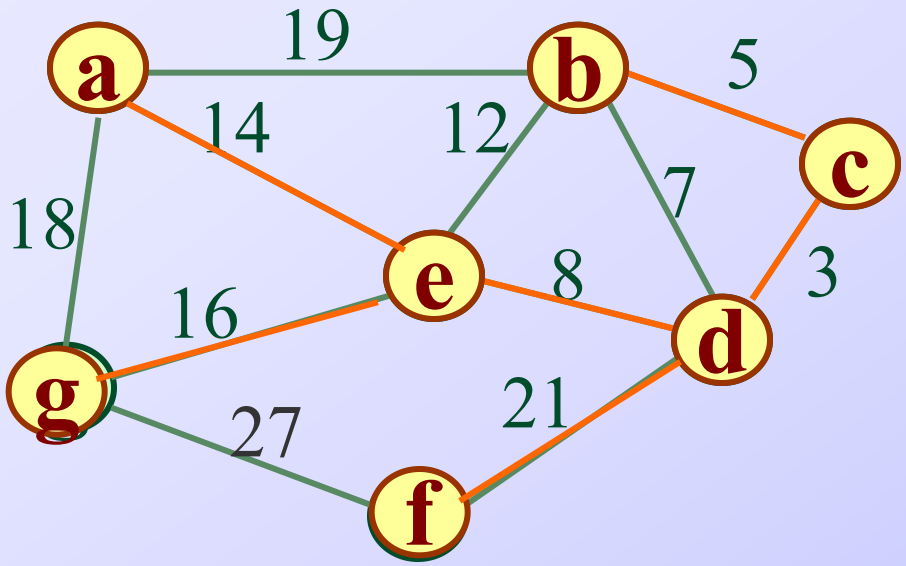
一般情况下所添加的顶点应满足下列条件：　在生成树的构造过程中，图中 n 个顶点分属两个集合：已落在生成树上的顶点集 U 和尚未落在生成树上的顶点集 V-U ，则应在所有连通U中顶点和V-U中顶点的边中选取权值最小的边。



设置一个辅助数组，对当前V－U集中的每个顶点，记录和顶点集U中顶点相连接的代价最小的边：

```
struct {
    VertexType  adjvex;  // U集中的顶点序号
    VRType      lowcost;  // 边的权值
} closedge[MAX_VERTEX_NUM];
```

例如:



	a	b	c	d	e	f	g
closedge	0	1	2	3	4	5	6
Adjvex		c	d	e	a	d	e
Lowcost		5	3	8	14	21	16

```
void MiniSpanTree_P(MGraph G, VertexType u) {
 //用普里姆算法从顶点u出发构造网G的最小生成树
  k = LocateVex ( G, u );
  for ( j=0; j<G.vexnum; ++j )  // 辅助数组初始化
     if (j!=k)
        closedge[j] = { u, G.arcs[k][j].adj };
  closedge[k].lowcost = 0;      // 初始，U＝{u}
  for (i=0; i<G.vexnum; ++i) {
     继续向生成树上添加顶点;
}
```

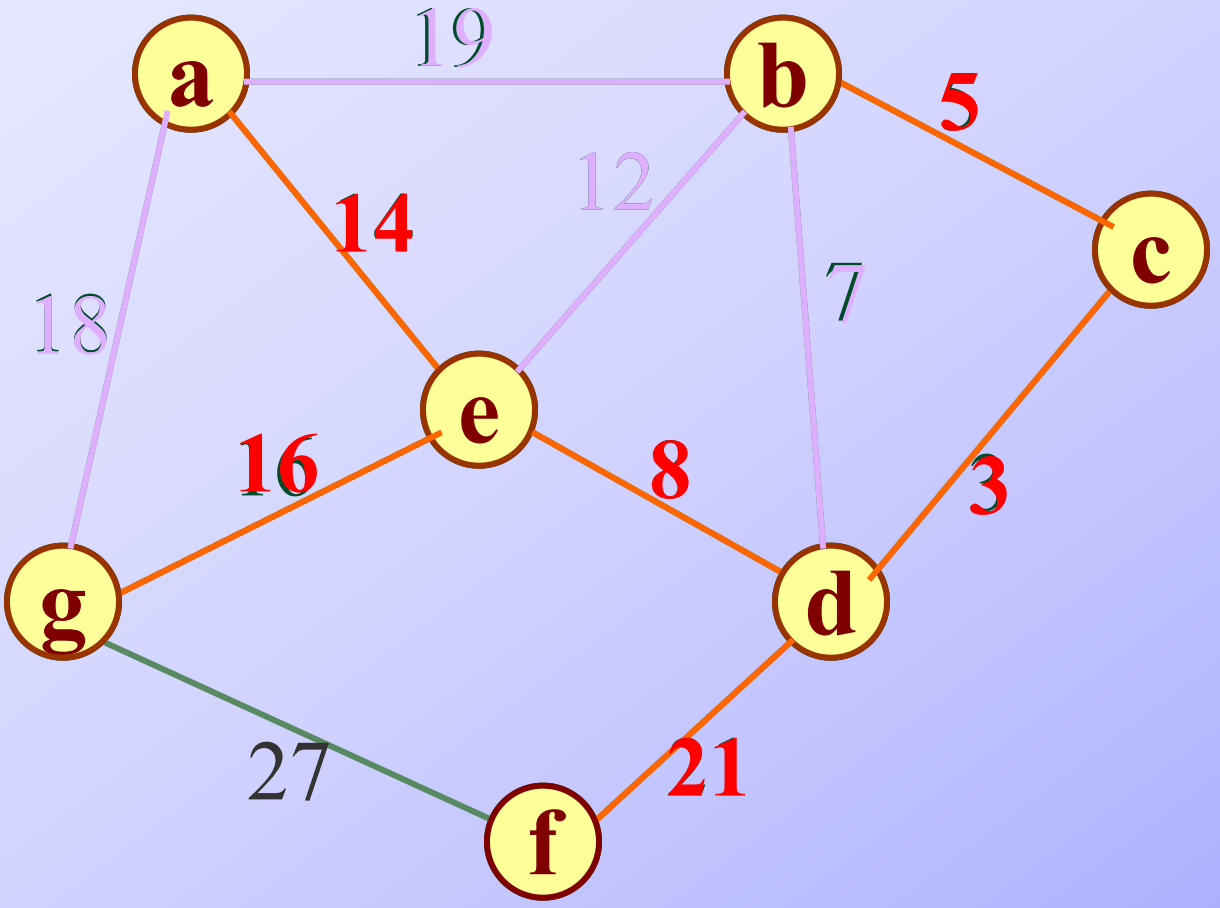
```
k = minimum(closedge);
                    // 求出加入生成树的下一个顶点(k)
printf(closedge[k].adjvex, G.vexs[k]);
                    // 输出生成树上一条边
closedge[k].lowcost = 0;    // 第k顶点并入U集
for (j=0; j<G.vexnum; ++j)
    //修改其它顶点的最小边
  if (G.arcs[k][j].adj < closedge[j].lowcost)
    closedge[j] = { G.vexs[k], G.arcs[k][j].adj };
```

克鲁斯卡尔算法的基本思想：

考虑问题的出发点：为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。

具体做法：先构造一个只含 n 个顶点的子图 SG，然后从权值最小的边开始，若它的添加不使SG 中产生回路，则在 SG 上加上这条边，如此重复，直至加上 n-1 条边为止。

例如:



算法描述:

构造非连通图 ST=(V,{ });

k = i = 0;　// k 计选中的边数

while (k<n-1) {

　++i;

　检查边集 E 中第 i 条权值最小的边(u,v);

　若(u,v)加入ST后不使ST中产生回路,

　则　输出边(u,v); 且　k++;

}

比较两种算法

算法名	普里姆算法	克鲁斯卡尔算法
时间复杂度	$O(n^2)$	$O(e\log e)$
适应范围	稠密图	稀疏图

§7.5 有向无环图及其应用

1. 拓扑排序

AOV网基本概念：

AOV网：如果用图中的顶点表示活动，边表示活动间的先后关系，则这样的有向图称为顶点活动网(Activity On Vertex network，简称AOV网)

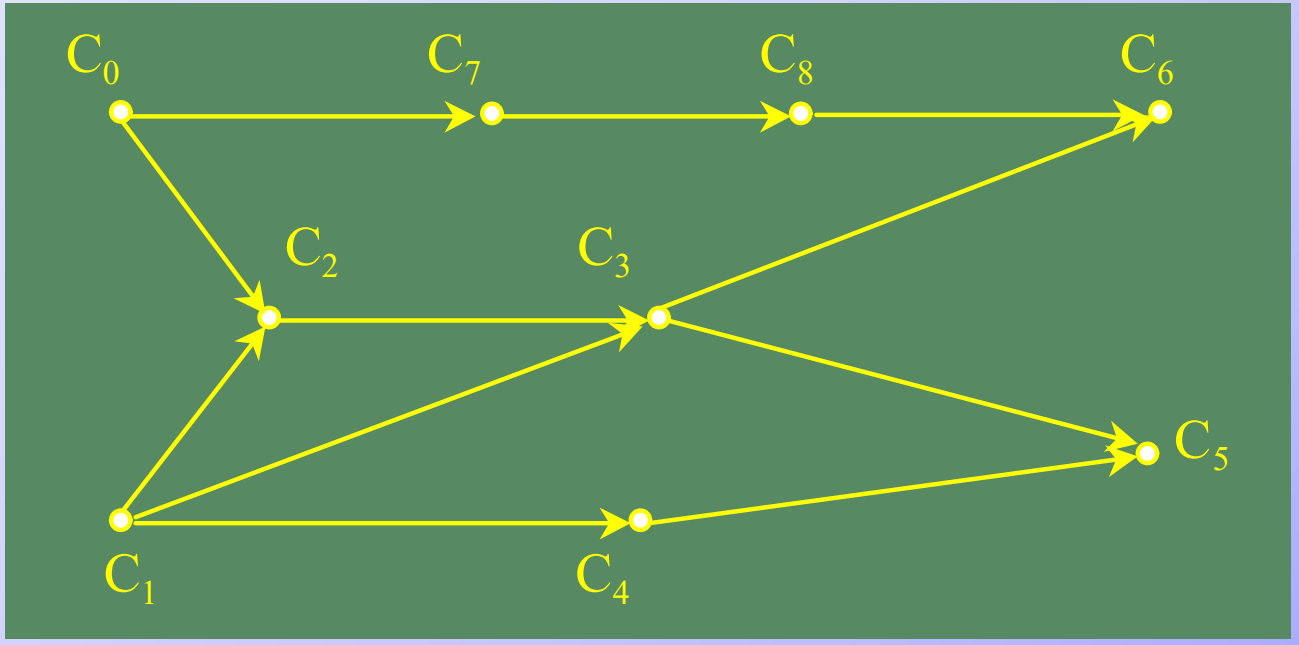
AOV网中的弧表示活动之间存在的制约关系

例如：

计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业，这时工程就是完成给定的学习计划，而活动就是学习课程，这些课程的名称与代号如表所示：

课程代号	课程名称	先修课程
C_0	高等数学	
C_1	程序设计语言	
C_2	离散数学	C_0,C_1
C_3	数据结构	C_1,C_2
C_4	算法语言	C_1
C_5	编译技术	C_3,C_4
C_6	操作系统	C_3,C_8
C_7	普通物理	C_0
C_8	计算机原理	C_7

在AOV网中，用顶点表示课程，有向边表示课程之间的优先关系，如果课程C_i是课程C_j的先修课，则在AOV网中必定存在一条有向边<C_i，C_j>。表中各课程的AOV网如下图所示



拓扑排序

对于一个AOV网，其所有顶点可以排成一个线性序列v_{i1}，v_{i2}，…，v_{in}，该线性序列具有以下性质：

如果在AOV网中，从顶点v_i到顶点v_j存在一条路径，则在线性序列中，顶点v_i一定排在顶点v_j之前。

具有这种性质的线性序列称为拓扑序列，构造拓扑序列的操作称为拓扑排序

例：对下图中的AOV网进行拓扑排序

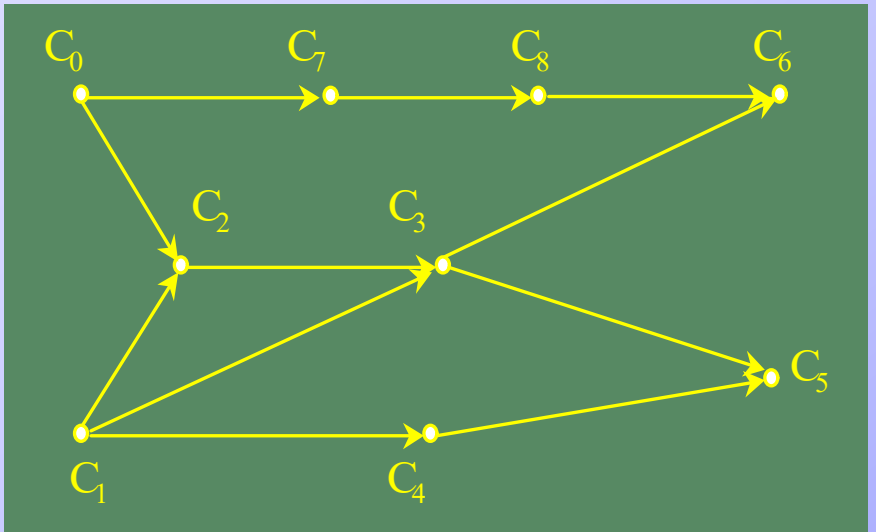
得到的一个拓扑序列：

C_0，C_1，C_2，C_4，C_3，C_5，C_7，C_8，C_6，

另外一个拓扑序列：

C_0，C_7，C_8，C_1，C_4，C_2，C_3，C_6，C_5

如果一个学生一学期只能选修一门课，则他必须按照某一个拓扑序列的次序学习，才能保证学习任何一门课时，其先修课程已学过。

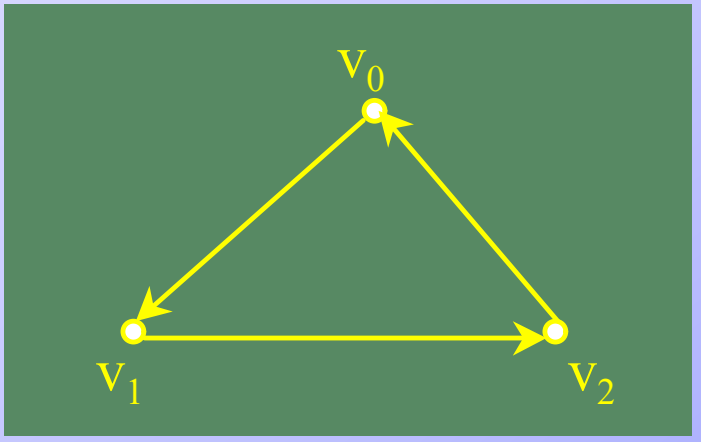


注意：

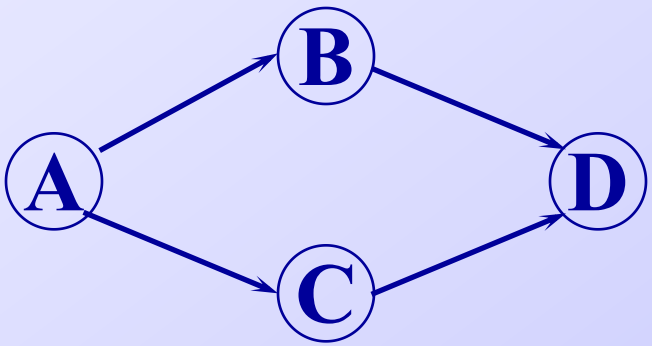
1、一个AOV网的拓扑序列不一定是唯一的。

2、假设AOV网代表一个工程，如果条件限制只能串行工作，则AOV网某一拓扑序列就是整个工程得以顺利完成的一种可行方案。 AOV网中一定不能出现回路(因为出现回路意味着，某些活动的开工是以自己工作的完成作为先决条件，这种现象称为死锁)

如图所示的AOV网，就无法把顶点排成满足拓扑序列条件的线性序列。



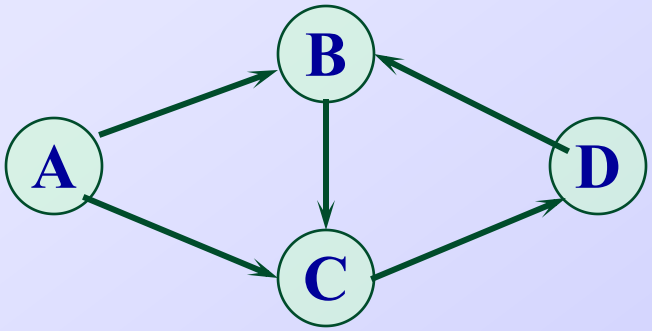
例如：对于下列有向图



可求得拓扑有序序列：

A B C D　或　A C B D

反之，对于下列有向图



不能求得它的拓扑有序序列。

因为图中存在一个回路 {B, C, D}

如何进行拓扑排序？

1. 从有向图中选取一个没有前驱的顶点，并输出之；

2. 从有向图中删去此顶点以及所有以它为尾的弧；

重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。

后一种情况说明图中存在环。

a b h c d g f e

在算法中需要用定量的描述替代定性的概念

没有前驱的顶点＝入度为零的顶点

删除顶点及以它为尾的弧＝弧头顶点的入度减1

拓扑排序算法：

设AOV网采用邻接表表示，边表为出边表。

定义一个indegree数组，存放各顶点的入度。

设置一个栈存储入度为0的顶点。拓扑排序前，先得到所有结点的入度，然后将所有入度为0的顶点压栈。

从栈顶取出一个顶点将其输出，由它的出边表可以得到以该顶点为起点的出边，将这些边终点的入度减1，即删除这些边。如果某条边终点的入度为0，则将该顶点入栈。

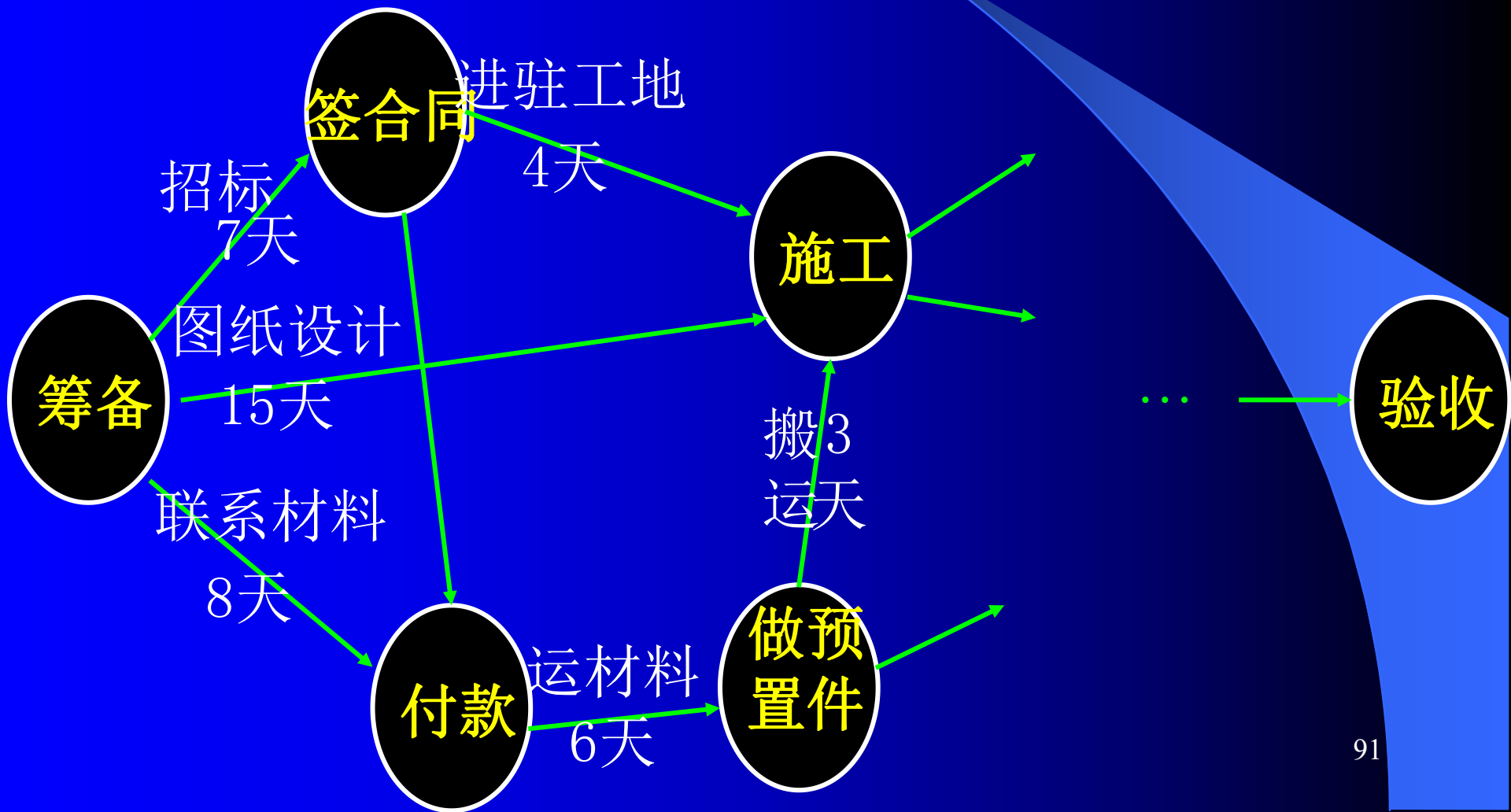
反复进行上述操作，直到栈为空，如果这时输出的顶点个数小于n，则说明该AOV网中存在回路，否则，拓扑排序正常结束。

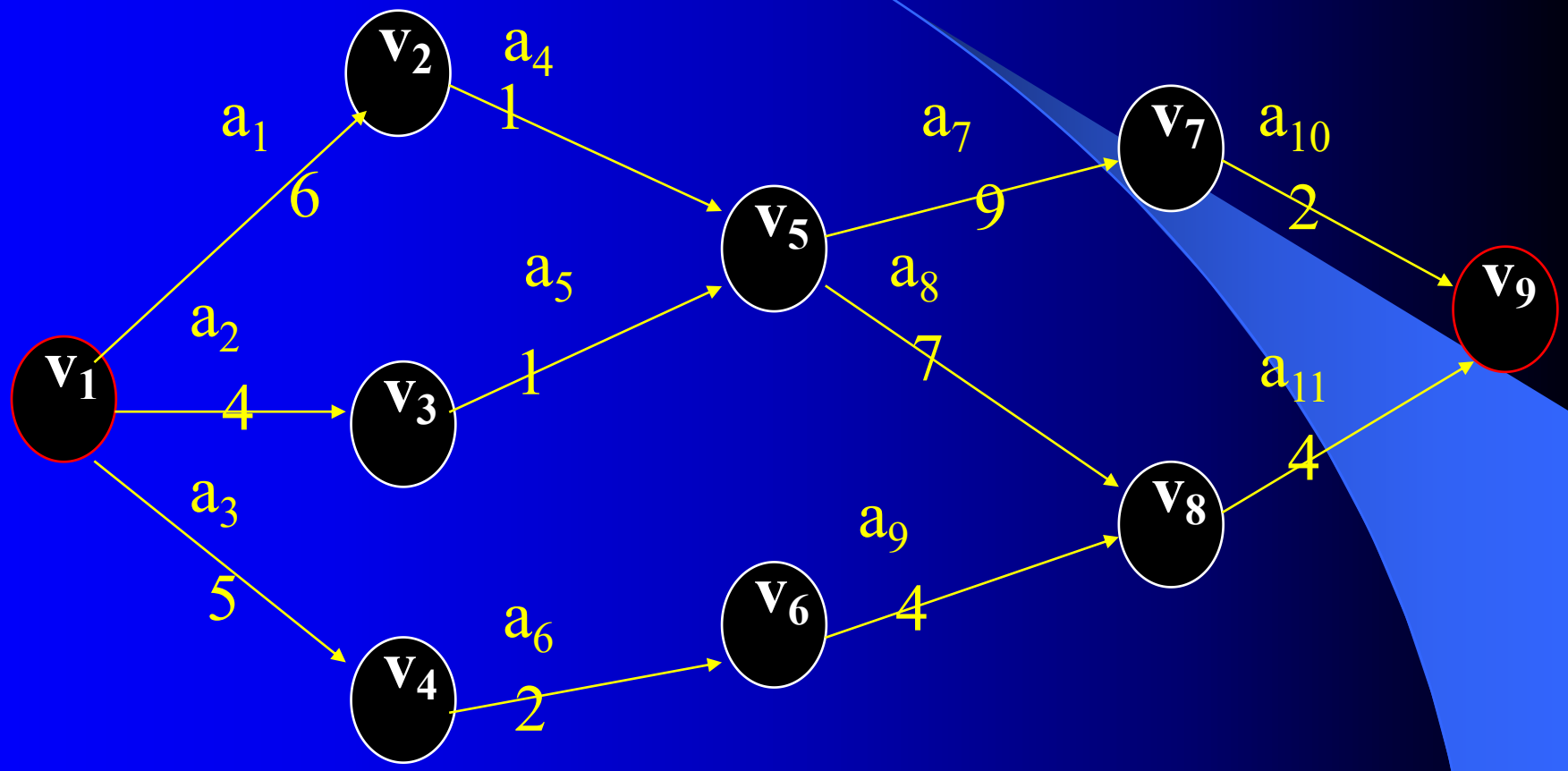
```
Status TopologicalSort(ALGraph G) {
//有向图G采用邻接表存储结构。若G无回路，则输出G
//的顶点的一个拓扑序列并返回OK，否则ERROR
FindInDegree(G,indegree);//求各顶点的入度
InitStack(S);
for(i=0;i<G.vexnum;++i)  //建零入度顶点栈S
  if(!indegree[i])  Push(S,i);  //入度为0顶点入栈
count=0;
while(!StackEmpty(S))  {
   ……
   //栈非空则对栈中入度为0顶点进行操作   }
if(count<G.vexnum)  return  ERROR;   //G有回路
else  return  OK;   }
```

```
Pop(S,i);
printf(i,G.vertics[i].data);++count;
//输出i号顶点并计数
for(p=G.vertics[i].firstarc;p;p=p->nextarc)
{   k=p->adjvex;
    //对i号顶点的每个邻接点的入度减1
    if(!(--indegree[k]))  Push(S,k);
    //如果入度减为0，则入栈
}
```

AOE网

AOE网为一个带权的有向无环图，其中，顶点表示事件，有向边表示活动，边上的权值表示活动持续的时间。




v1
v2
v3
v4
v5
v6
v7
v8
v9
a1
6
a2
4
a3
5
a4
1
a5
1
a6
2
a7
9
a8
7
a9
4
a10
2
a11
4

2. 关键路径

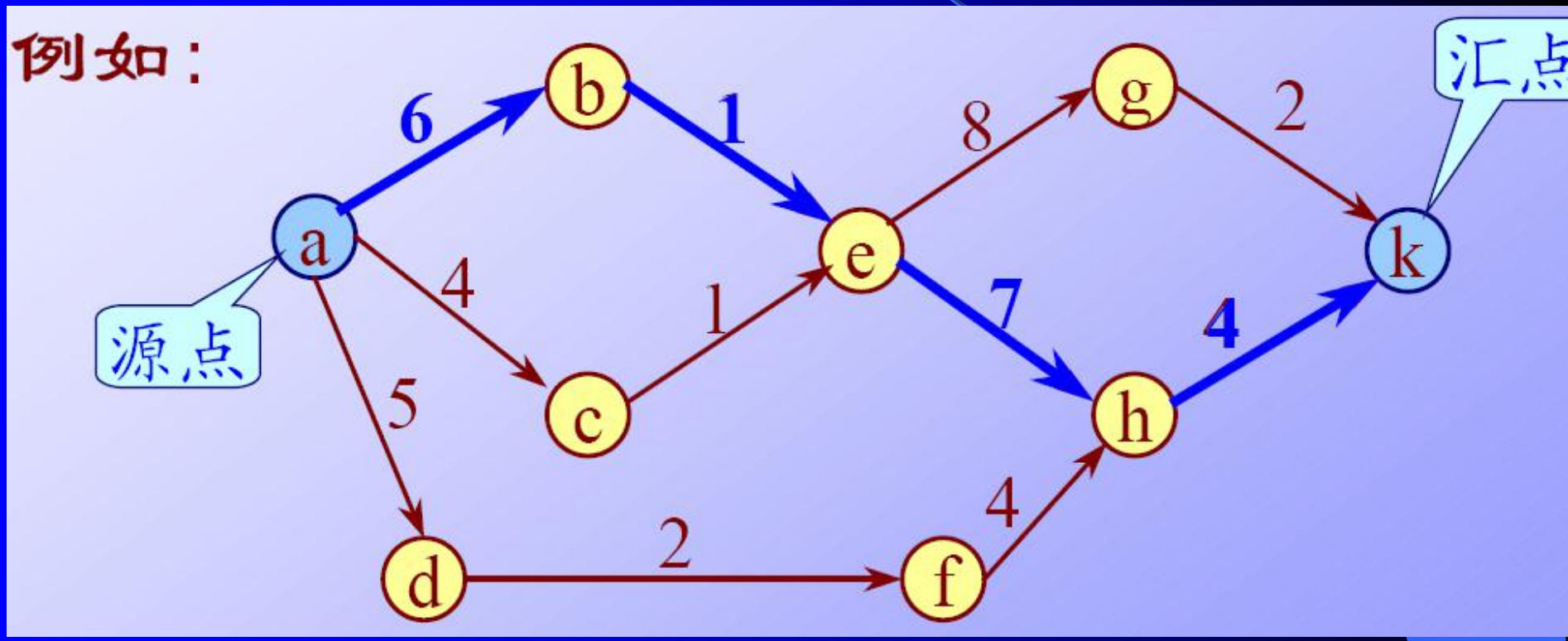
问题：

假设以有向网表示一个施工流图，弧上的权值表示完成某活动所需时间。

问：哪些活动是“关键活动”？

即：哪些活动将影响整个工程的完成期限的。

整个工程完成的时间为：从有向图的源点到汇点的最长路径。



“关键活动”指的是：该弧上的权值增加将使有向图上的最长路径的长度增加。

关键路径的特点：

1）关键路径的长度（路径上的边的权值之和）为完成整个工程所需要的最短时间。

2）关键路径的长度变化（即任意关键活动的权值变化）将影响整个工程的进度，而其他非关键活动在一定范围内的变化不会影响工期。

求关键活动的思路：

e（i） — 活动a_i 的最早开始时间

l（i） — 活动a_i 的最迟开始时间

若l（i）－e（i）=0，则说明活动a_i 为一个关键活动

ee(k) — 事件k 的最早发生时间

le(k) — 事件k 的最迟发生时间

结论：

事件k的最早发生时间ee(k) → 活动的a_i 最早开始时间e(i)

事件k的最迟发生时间le(k) → 活动的a_i 最迟开始时间l(i)

↓

求 e(i) = l(i)

求关键路径

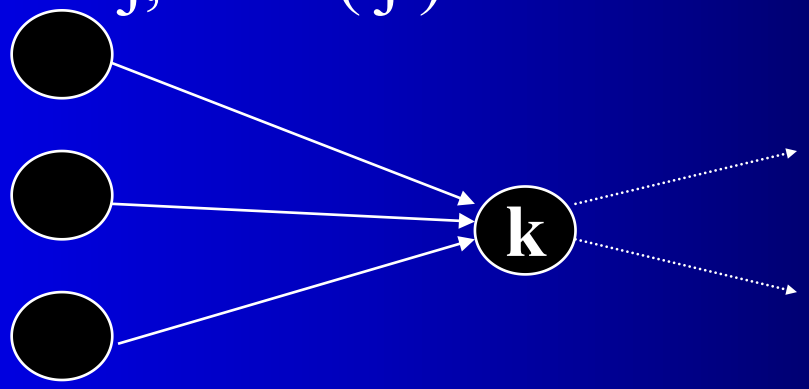
1. 计算事件k的最早发生时间ee(k)

所谓事件k的最早发生时间ee(k)是指从源点到顶点k的最大路径长度.

计算方法:

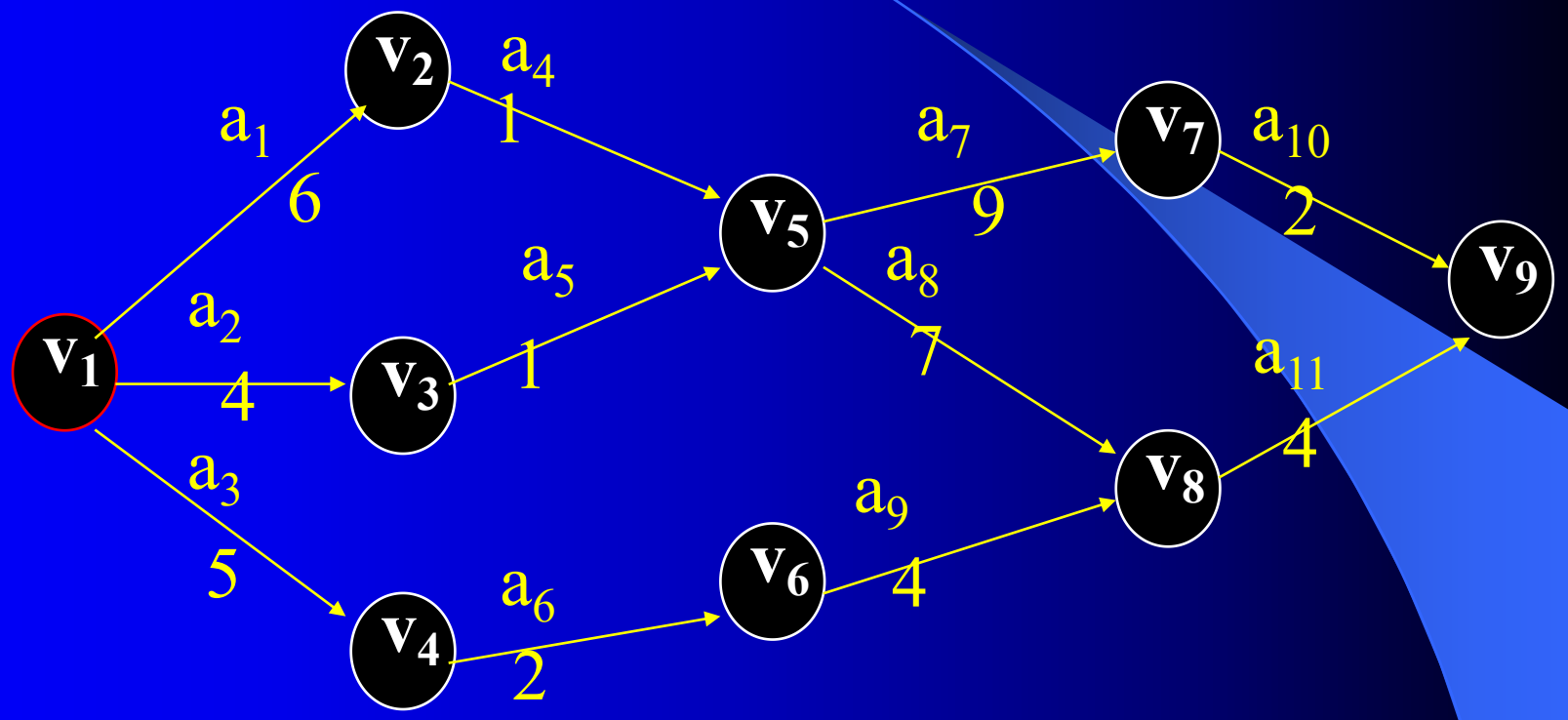
$$ee(1) = 0$$

$$ee(k) = \underset{\langle j,k\rangle \in P(j)}{MAX}\{ ee(j) + \langle j,k\rangle \text{的权}\}$$



ee(k) :

1	2	3	4	5	6	7	8	9
0	6	4	5	7	7	16	14	18



计算方法:

ee(1) = 0

ee(k) = MAX{ ee(j) + <j, k>的权 }

<j,k>∈P(j)

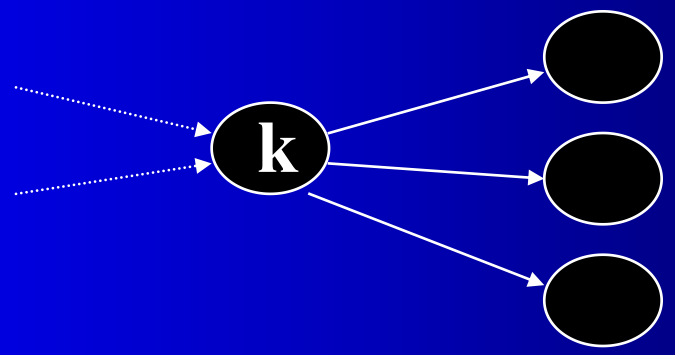
2. 计算事件k的最迟发生时间le(k)

所谓事件k 的最迟发生时间le(k)是指不影响整个工期的前提下事件k 必须发生的最晚时间。

计算方法:

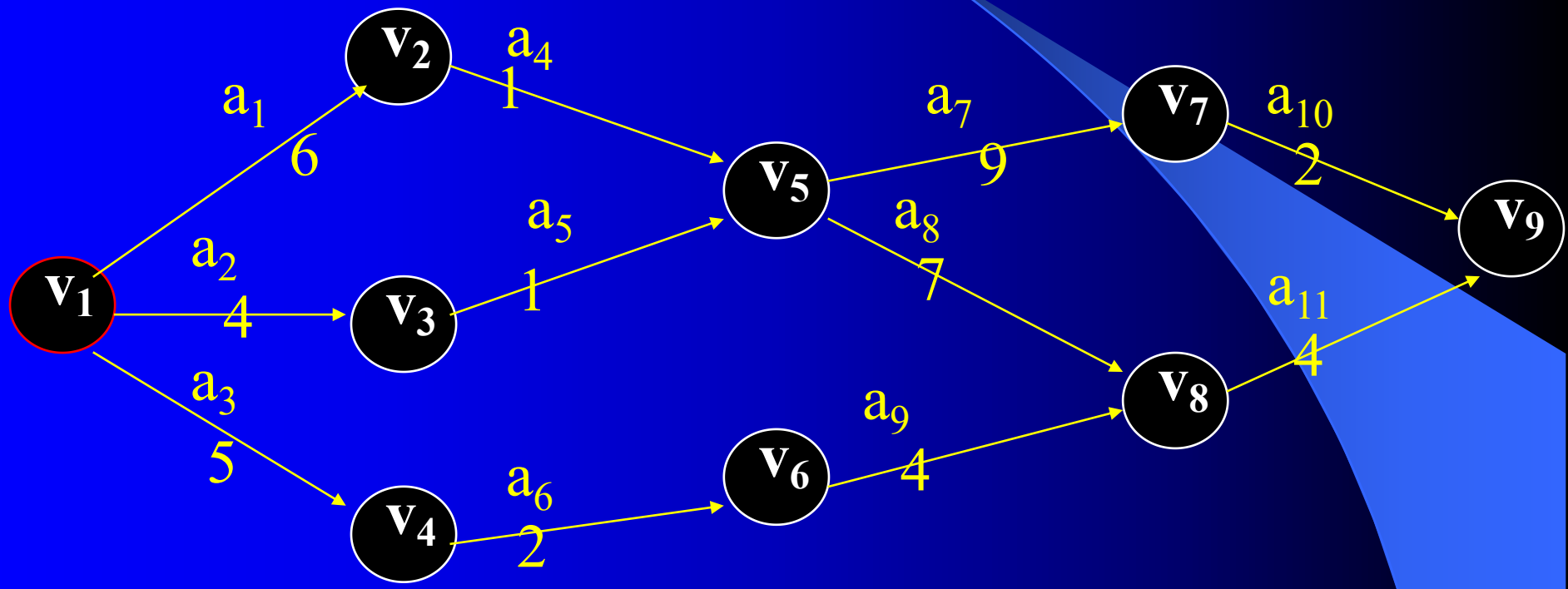
$$le(n) = ee(n)$$

$$le(k) = \underset{<k,j>\in S(k)}{MIN}\{ le(j) - <k,j>\text{的权} \}$$



le(k) :

0	6	6	8	7	10	16	14	18
1	2	3	4	5	6	7	8	9



计算方法:

$$le(n) = ee(n)$$

$$le(k) = \underset{<k,j>\in S(k)}{MIN}\{ le(j) - <k,j>\text{的权} \}$$

3. 计算活动i的最早开始时间e(i)

所谓活动i 的最早开始时间e(i) 是事件k发生的最早时间，即只有事件k 发生了，活动i 才能开始。

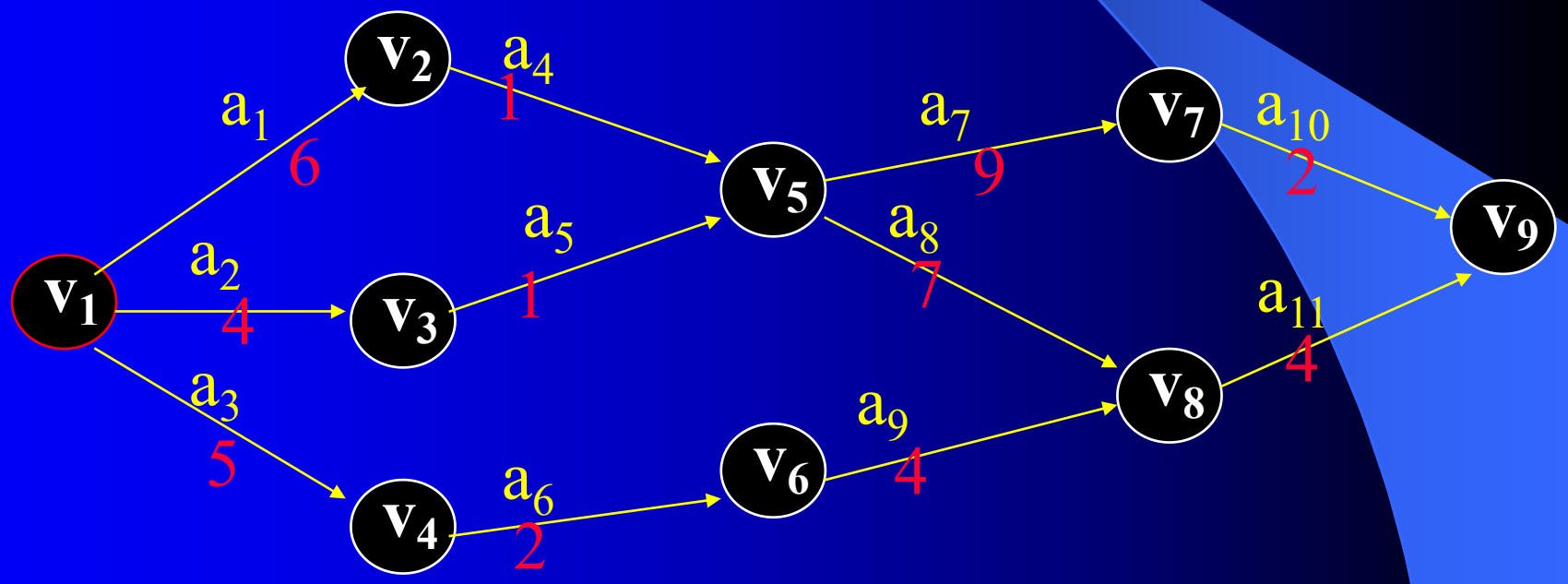
计算方法：　　$e(i) = ee(k)$

ee(k) :

1	2	3	4	5	6	7	8	9
0	6	4	5	7	7	16	14	18

e(i) :

1	2	3	4	5	6	7	8	9	10	11
0	0	0	6	4	5	7	7	7	16	14

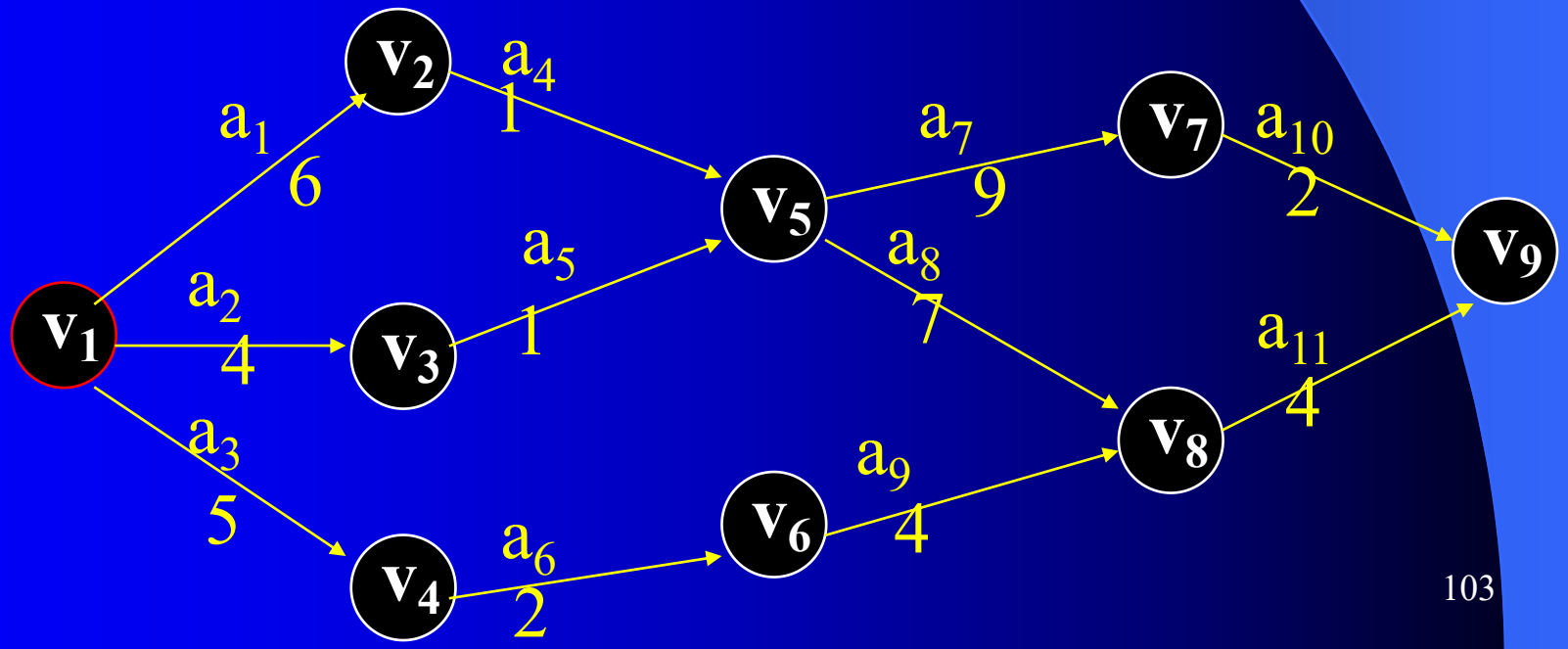


计算方法： e(i) = ee(k)

4. 计算活动i 的最迟开始时间l（i）

所谓活动i 的最迟开始时间l（i） 是指不推迟整个工期的前提下活动i 开始的最晚时间。

计算方法： $l(i) = le(j) - \langle k,j \rangle$的权

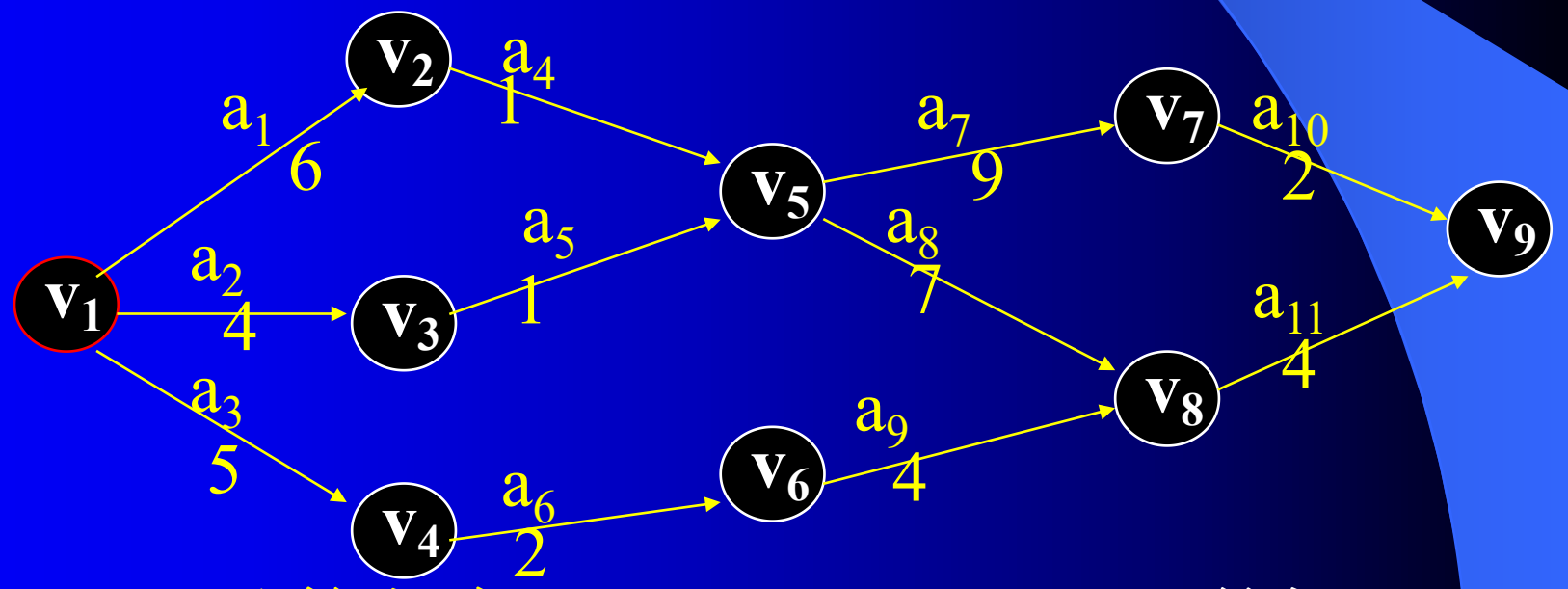


le(j) :

0	6	6	8	7	10	16	14	18
1	2	3	4	5	6	7	8	9

l(i) :

0	2	3	6	6	8	7	7	10	16	14
1	2	3	4	5	6	7	8	9	10	11



计算方法: l(i) = le(j) −<k,j>的权

5. 求出关键活动与关键路径

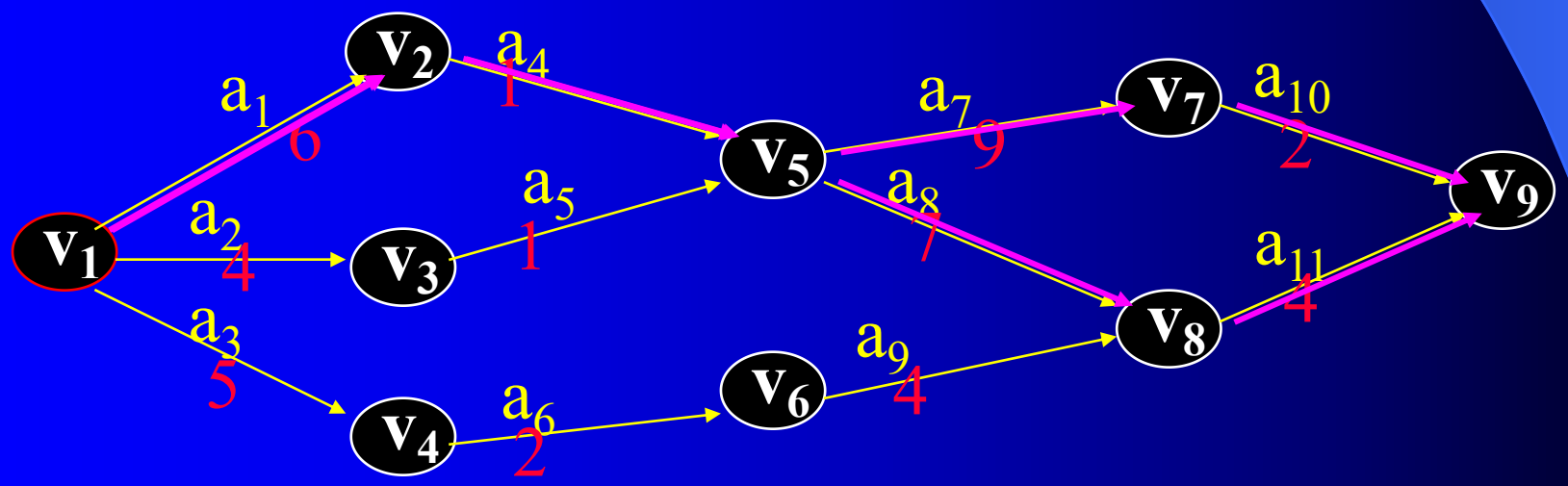
计算方法：　　l(i) = e(i)

e(i)：

1	2	3	4	5	6	7	8	9	10	11
0	0	0	6	4	5	7	7	7	16	14

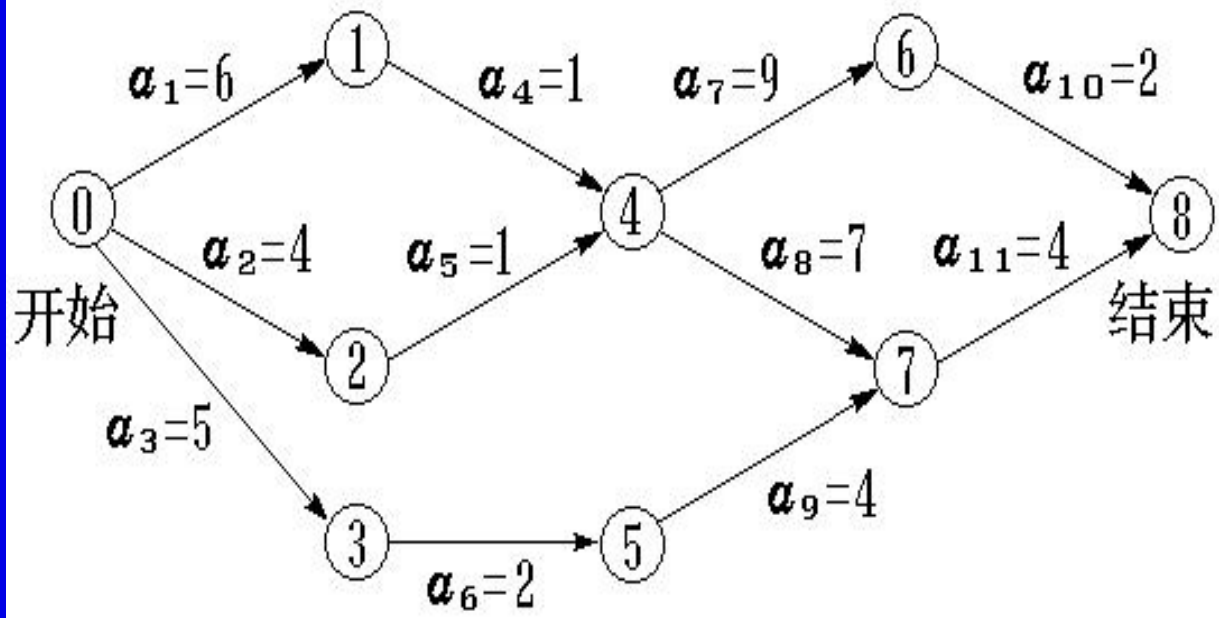
l(i)：

1	2	3	4	5	6	7	8	9	10	11
0	2	3	6	6	8	7	7	10	16	14
a_1			a_4			a_7	a_8		a_{10}	a_{11}



关键路径的长度为18 个时间单位。

事件	$Ve[i]$	$Vl[i]$
V_0	0	0
V_1	6	6
V_2	4	6
V_3	5	8
V_4	7	7
V_5	7	10
V_6	16	16
V_7	14	14
V_8	18	18



(a)

边	<0,1>	<0,2>	<0,3>	<1,4>	<2,4>	<3,5>	<4,6>	<4,7>	<5,7>	<6,8>	<7,8>
活动	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
e	0	0	0	6	4	5	7	7	7	16	14
l	0	2	3	6	6	8	7	7	10	16	14
$l-e$	0	2	3	0	2	3	0	0	3	0	0
关键	是			是			是	是		是	是

算法的实现要点

显然，求ve的顺序应该是按拓扑有序的次序而求vl的顺序应该是按拓扑逆序的次序；因为拓扑逆序序列即为拓扑有序序列的逆序列，

因此，应该在拓扑排序的过程中，另设一个“栈”记下拓扑有序序列。

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。

2. 熟练掌握图的两种搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。

在学习中应注意图的遍历算法与树的遍历算法之间的类似和差异。

3. 应用图的遍历算法求解各种简单路径问题。

4. 理解教科书中讨论的各种图的算法。